



acm International Collegiate
Programming Contest

IBM

event
sponsor

数据结构

Ciallo~(∠・ω<)〰☆

目 录

数据结构	3
基本数据结构	3
链表	3
单链表	3
双链表	5
栈	7
表达式计算	8
单调栈	9
队列	11
单调队列	12
滑动窗口	12
最大子段和	13
堆	14
二叉堆	14
左偏树	16
配对堆	17
哈希	19
开放寻址法	19
拉链法	20
并查集	22
边带权并查集	22
拓展域并查集	24
可撤销并查集	26
带删除并查集	28
可持久化并查集	30
分块	32
数列分块	32
值域分块	37
树状数组	40
一维树状数组	40
二维树状数组	42
权值树状数组	44
静态区间颜色数	47

线段树	49
单点修改, 区间查询	50
区间修改, 区间查询, lazy标记	56
权值线段树	59
线段树上二分	62
可持久化线段树	64
霍夫曼树	72
平衡树	76
Splay树	76
笛卡尔树	82

数据结构

基本数据结构

链表

单链表

```
1  const int N = 100010;
2  //head 表示头节点的下标
3  //e[i] 表示节点i的值
4  //ne[i] 表示节点i的next指针是多少
5  //idx 储存当前已经用到了哪个点
6  int head, e[N], ne[N], idx;

7  //初始化
8  void init() {
9      head = -1;
10     idx = 0;
11 }
```

```
1  //将x插到头节点
2  void add_to_head(int x) {
3      e[idx] = x;
4      ne[idx] = head; //x指向开头
5      head = idx;     //x的位置变为新的开头
6      idx++;
7  }
```

```
1  //将x插到下标是k的点的后面(注意下标从0开始)
2  void add(int k, int x) {
3      e[idx] = x;
4      ne[idx] = ne[k]; //x指向k的后一位
5      ne[k] = idx;     //k再指向x
6      idx++;
7  }
```

```
1  //将下标是k的点的后面一个点删掉
2  void remove(int k) {
3      ne[k] = ne[ne[k]]; //原来指向k的后位改为指向k后位的后位
4  }
```

```
1  //遍历链表
2  for(int i = head; i != -1; i = ne[i]){
3      cout << e[i] << " ";
4  }
```

```

1 //https://www.acwing.com/problem/content/828/
2 #include <bits/stdc++.h>
3 using namespace std;
4 const int N = 100005;
5 int e[N], ne[N], idx, h = -1;
6
7 void add_to_head(int x) {
8     e[idx] = x;
9     ne[idx] = h;
10    h = idx++;
11 }
12
13 void add(int k, int x) {
14     e[idx] = x;
15     ne[idx] = ne[k];
16     ne[k] = idx++;
17 }
18
19 void del(int k) {
20     ne[k] = ne[ne[k]];
21 }
22
23 int main() {
24     int t; cin >> t;
25     for (int i = 1; i <= t; i++) {
26         char op; cin >> op;
27         if (op == 'I') {
28             int k, x; cin >> k >> x;
29             add(k - 1, x); //第k个数，下标为k-1
30         }
31         if (op == 'H') {
32             int x; cin >> x;
33             add_to_head(x);
34         }
35         if (op == 'D') {
36             int k; cin >> k;
37             if (k == 0) h = ne[h]; //k == 0 表示删除头结点
38             else del(k-1);
39         }
40     }
41
42     for (int i = h; i != -1; i = ne[i]) cout << e[i] << " ";
43
44     return 0;
45 }

```

双链表

```
1  const int N = 100010;
2  int e[N], l[N], r[N], idx;
3
4  //初始化
5  void init() {
6      //0表示左端点, 1表示右端点
7      r[0] = 1, l[1] = 0;
8      idx = 2;    //从2开始
9  }
```

```
1  //在下标为k的右边插入值x
2  void addr(int k, int x) {
3      e[idx] = x;
4      r[idx] = r[k]; //x的右边指向k的下一位
5      l[idx] = k;    //x的左边指向k
6      l[r[k]] = idx; //k的下一位的左边指向x (注意与下一条语句顺序不能写反)
7      r[k] = idx;   //k的右边指向x
8  }
```

```
1  //在下标为k的左边插入值x
2  void addl(int k, int x) {
3      addr(l[k], x); //相当于在k左边一个数的右边插入x
4  }
```

```
1  //删除下标为k的点
2  void remove(int k) {
3      r[l[k]] = r[k]; //让原来右边指向k的指针 指向k的右边
4      l[r[k]] = l[k]; //让原来左边指向k的指针 指向k的左边
5  }
```

[APIO/CTSC2007\(数据备份\)](#)

问题转化为差分数组 $a_1 \sim a_{n-1}$ 中选则各不相邻的k个数

```
1  #include <iostream>
2  #include <queue>
3  using ll = long long;
4  using namespace std;
5  const ll inf = 1e18;
6  const int N = 100005;
7  int n, k;
8  ll a[N];
9  ll ne[N], la[N], s[N]; //双链表
10 bool del[N]; //标记当前节点是否已经删除
11 ll ans;
12
```

```

13 struct Edge{
14     ll id,x;
15     bool operator < (const Edge &e) const {
16         return x > e.x;//优先队列小根堆
17     }
18 };
19
20 int main(){
21     cin >> n >> k;
22     priority_queue<Edge>pq;
23     for(int i = 1;i <= n;i++){
24         cin >> a[i];
25         if(i > 1) s[i-1] = a[i] - a[i-1];
26     }
27
28     s[0] = s[n] = inf;//左右两边设置哨兵
29     for(int i = 1;i < n;i++){
30         pq.push({i,s[i]});
31         ne[i] = i+1;
32         la[i] = i-1;
33     }
34
35     while(k--){
36         while(del[pq.top().id]) pq.pop();//若该点已经被删除，则直接弹出
37         auto [id,x] = pq.top();
38         pq.pop();
39         ans += x;//选择当前节点
40
41         //删除当前节点和左右两边的节点，在当前节点位置新建一个节点
42         //若未来选了该新节点，则相当于不选当前节点，选择当前节点左右两边的节点
43         s[id] = s[la[id]] + s[ne[id]] - s[id];
44         Edge e = {id,s[id]};
45         pq.push(e);
46
47         del[ne[id]] = del[la[id]] = 1;//删除左右两边的节点
48         la[id] = la[la[id]];
49         ne[id] = ne[ne[id]];
50         ne[la[id]] = id;
51         la[ne[id]] = id;
52     }
53     cout << ans;
54 }

```

栈

先进后出

```
1  #include <iostream>
2  using namespace std;
3  const int N = 100010;
4  //tt表示栈尾
5  int stk[N], tt;
6
7  //插入
8  stk[++tt] = x;
9
10 //弹出
11 tt--;
12
13 //判断栈是否为空
14 if (tt > 0) not empty
15 else empty
16
17 //栈顶
18 stk[tt];
```

```
1  //带最小值的栈
2  //新建一个栈B,每次A入栈x时,B入栈min(B.top(),x);
3  stack<int>A,B;
4
5  void push(int x){
6      A.push(x);
7      if(B.empty()) B.push(x);
8      else B.push(min(x,B.top()));
9  }
10
11 void pop(){
12     A.pop();
13     B.pop();
14 }
15
16 int top(){
17     return A.top();
18 }
19
20 int getmin(){
21     return B.top();
22 }
```

```
1  //火车进站方案 https://www.acwing.com/problem/content/131/
2  #include <iostream>
```

```

3  #include <vector>
4  #include <stack>
5  using namespace std;
6  const int N = 30;
7  int n, rest;
8  vector<int> p; //p存已经出站的序列
9  stack<int> st; //栈
10
11 void dfs(int u){
12     if(rest >= 20) return;
13     if(p.size() == n){ //边界条件，如果全部出站则输出序列
14         rest++;
15         for(auto &x:p) cout << x;
16         cout << endl;
17         return;
18     }
19     if(st.size()){ //情况一：栈中元素出站
20         p.push_back(st.top());
21         st.pop();
22         dfs(u);
23         st.push(p.back()); //还原现场
24         p.pop_back();
25     }
26     if(u <= n){ //情况二：火车入栈
27         st.push(u);
28         dfs(u+1);
29         st.pop(); //还原现场
30     }
31 }
32
33 int main(){
34     cin >> n;
35     dfs(1);
36     return 0;
37 }

```

表达式计算

中缀表达式：A op B

前缀表达式：op A B （波兰式）

后缀表达式：A B op （逆波兰式）

后缀表达式求值

后缀表达式对于计算机来讲容易实现

建立一个用于存储数的栈，逐一扫描该后缀表达式中的元素

1. 如果遇到一个数，则把该数入栈。
2. 如果遇到运算符，则取出栈顶两个数进行计算，把结果入栈。

扫描完成后，栈中恰好剩下一个数，即为该后缀表达式的答案

```
1 if(c[k]>='0' && c[k]<='9') q.push(c[k]-'0'); //假设表达式都是1位数
2 if(c[k]=='-') i=q.top(),q.pop(),j=q.top(),q.pop(), q.push(j-i);
3 if(c[k]=='+') i=q.top(),q.pop(),j=q.top(),q.pop(), q.push(j+i);
4 if(c[k]=='*') i=q.top(),q.pop(),j=q.top(),q.pop(), q.push(j*i);
5 if(c[k]=='/') i=q.top(),q.pop(),j=q.top(),q.pop(), q.push(j/i);
```

单调栈

```
1 //输出每个数左边第一个比它小的数，如果不存在则输出-1;
2 #include <iostream>
3 #include <stack>
4 using namespace std;
5 const int N = 100005;
6 int n,a[N];
7 stack<int>sk;
8 int ans[N];
9
10 int main(){
11     cin >> n;
12     for(int i = 1;i <= n;i++) { cin >> a[i]; }
13
14     for(int i = 1;i <= n;i++){
15         while(sk.size() && sk.top() >= a[i]) sk.pop();
16         if(sk.size()) ans[i] = sk.top();
17         else ans[i] = -1;
18
19         sk.push(a[i]);
20     }
21
22     for(int i = 1;i <= n;i++){ cout << ans[i] << ' '; }
23 }
```

直方图中最大的矩形

```
1 //https://www.acwing.com/problem/content/133/
2 #include <iostream>
3 #include <stack>
4 using namespace std;
5 using ll = long long;
```

```

6  const int N = 100005;
7  ll a[N];
8  ll l[N], r[N]; // l[i] 记录左边最后一个大于等于本身的下标, r[i] 记录右边最后一个大于等于本身的下标
9  // 对于每一个高度 a[i], 能组成的最大面积为 a[i] * (r[i] - l[i] + 1)
10
11 int main(){
12     int n;
13     while(cin >> n, n){
14         a[0] = a[n+1] = -1; // 在边界设置哨兵
15         for(int i = 1; i <= n; i++) cin >> a[i];
16
17         stack<ll> sk;
18         for(int i = 0; i <= n+1; i++){
19             while(sk.size() && a[sk.top()] >= a[i]) sk.pop();
20             if(sk.size()) l[i] = sk.top() + 1;
21             else l[i] = i;
22             sk.push(i);
23         }
24
25         sk = stack<ll>();
26         for(int i = n+1; i >= 0; i--){
27             while(sk.size() && a[sk.top()] >= a[i]) sk.pop();
28             if(sk.size()) r[i] = sk.top() - 1;
29             else r[i] = i;
30             sk.push(i);
31         }
32
33         ll ans = 0;
34         for(int i = 1; i <= n; i++){
35             ans = max(ans, a[i] * (r[i] - l[i] + 1));
36         }
37         cout << ans << endl;
38     }
39     return 0;
40 }

```

[U478856 区间最值和 - 洛谷](#)

求所有区间的最大值之和, $1 \leq n \leq 10^5, 1 \leq a[i] \leq 10^9$ 。

$$\sum_{1 \leq l \leq r \leq n} \max(a[l], r])$$

单独考虑每个位置 i 的贡献, 找到左边第一个大于 $a[i]$ 的位置 l 和右边第一个大于等于 $a[i]$ 的位置 r ,

则位置 i 的贡献为 $a[i] \times (i - l) \times (r - i)$ 。取左闭右开区间或左开右闭区间防止区间所有数相同时会重复计算一次。

```

1  n = int(input())
2  a = [10 ** 10] + list(map(int, input().split())) + [10 ** 10]
3

```

```

4 left = [0] * (n + 1)
5 right = [0] * (n + 1)
6
7 stk = [0]
8 for i in range(1, n + 1):
9     while stk and a[stk[-1]] <= a[i]: # 一边 > a[i]
10         stk.pop()
11     left[i] = stk[-1]
12     stk.append(i)
13
14 stk = [n + 1]
15 for i in range(n, 0, -1):
16     while stk and a[stk[-1]] < a[i]: # 一边 >= a[i]
17         stk.pop()
18     right[i] = stk[-1]
19     stk.append(i)
20
21
22 ans = 0
23 for i in range(1, n + 1):
24     l = i - left[i]
25     r = right[i] - i
26     ans += a[i] * l * r
27     #如果题目要求贡献还要乘以区间长度，则只需这里改为ans += a[i] * l * r * (l + r) // 2
28
29 print(ans)

```

队列

ql	←	←	←	←	qr
hh					tt

先进先出

```

1 //双端队列
2 //hh表示队首front，tt表示队尾back
3 int q[N], hh = 0, tt = -1;
4
5 //入队(队尾)
6 q[++tt] = x;
7
8 //弹出(队首)
9 hh++;
10

```

```

11 //判断队列是否非空
12 if (hh <= tt) return 1;
13
14
15 q[hh]//队首元素
16 q[tt]//队尾元素

```

单调队列

滑动窗口

窗口位置	最小值	最大值
[1 3 -1] -3 5 3 6 7	-1	3
1 [3 -1 -3] 5 3 6 7	-3	3
1 3 [-1 -3 5] 3 6 7	-3	5
1 3 -1 [-3 5 3] 6 7	-3	5
1 3 -1 -3 [5 3 6] 7	3	6
1 3 -1 -3 5 [3 6 7]	3	7

此处的 "队列" 跟普通队列的一大不同就在于可以从队尾进行操作, STL 中有类似的数据结构 deque

```

1 //滑动窗口之最值
2 #include <iostream>
3 using namespace std;
4 const int N = 1000010;
5 int a[N], q[N]; //a[N]为数组, q[N]队列存下标
6 int main() {
7     int n, k;
8     cin >> n >> k;
9     for (int i = 1; i <= n; i++) scanf("%d", &a[i]);
10
11     int hh = 0, tt = -1;
12     for (int i = 1; i <= n; i++){//hh<=tt判断队列是否为空, 相当于!q.empty()或q.size()
13         if (hh <= tt && i-k >= q[hh]) hh++;//i-k>=q[hh]窗口已经形成,则每次循环队首右
移
14         while (hh <= tt && a[q[tt]] > a[i]) tt--;//求最小, 把队列中>a[i]的数都弹出
15         q[++tt] = i;
16         if (i >= k) printf("%d ", a[q[hh]]);//队首即为最小值
17     }
18     puts("");
19
20     hh = 0, tt = -1;
21     for (int i = 1; i <= n; i++){
22         if (hh <= tt && i-k >= q[hh]) hh++;

```

```

23     while (hh <= tt && a[q[tt]] < a[i]) tt--; //求最大类似
24     q[++tt] = i;
25     if (i >= k) printf("%d ", a[q[hh]]);
26 }
27 return 0;
28 }

```

```

1 //STL的deque实现，比数组模拟要慢
2 #include <iostream>
3 #include <deque>
4 using namespace std;
5 const int N = 1000006;
6 int a[N];
7
8 int main(){
9     int n,k; cin >> n >> k;
10    for(int i = 1; i <= n; i++) {cin >> a[i];}
11
12    deque<int> dq;
13    for(int i = 1; i <= n; i++){
14        if(dq.size() && i - k >= dq.front()) dq.pop_front();
15        while(dq.size() && a[dq.back()] >= a[i]) dq.pop_back();
16        dq.push_back(i);
17        if(i >= k) cout << a[dq.front()] << ' ';
18    }
19    cout << '\n';
20
21    dq.clear();
22    for(int i = 1; i <= n; i++){
23        if(dq.size() && i - k >= dq.front()) dq.pop_front();
24        while(dq.size() && a[dq.back()] <= a[i]) dq.pop_back();
25        dq.push_back(i);
26        if(i >= k) cout << a[dq.front()] << ' ';
27    }
28 }

```

最大子段和

给定一个长度为 n 的整数序列，从中找出一段长度不超过 m 的连续子序列，使得子序列中所有数的和最大。

最优策略是，下标递增，对应前缀和也递增

```

1 #include <iostream>
2 #include <cstring>
3 using namespace std;
4 const int N = 300005;
5 int n,m,a[N];

```

```

6  int q[N],hh,tt;//tt = -1; q[++tt] = 0;
7
8  int main(){
9      cin >> n >> m;
10     for(int i = 1;i <= n;i++){
11         cin >> a[i];
12         a[i] += a[i-1];
13     }
14
15     int ans = 0x80000000; //初始化为负无穷
16     for(int i = 1;i <= n;i++){
17         ans = max(ans,a[i]-a[q[hh]]);
18         if(hh <= tt && i - q[hh] >= m) hh++;
19         while(hh <= tt && a[q[tt]] >= a[i]) tt--;//单调递增队列,队首a[q[hh]]为最小值
20         q[++tt] = i;
21     }
22     cout << ans;
23
24     return 0;
25 }

```

堆

堆是一棵完全二叉树，其每个节点都有一个键值，且每个节点的键值都大于等于(小于等于)其父亲的键值。

堆主要支持的操作有：插入一个数、查询最小值、删除最小值、合并两个堆、减小一个元素的值。

一些功能强大的堆（可并堆）还能（高效地）支持 merge 等操作。

一些功能更强大的堆还支持可持久化，也就是对任意历史版本进行查询或者操作，产生新的版本。

二叉堆

//下标从1开始比较方便：第k个数，左儿子为2k，右儿子为2k+1

操作	实现
1.插入一个数	heap[++size] = x; up(size);
2.求集合当中最小值	heap[1];
3.[删除最小值][将第一个数等于最后一个数，删掉最后一个数，再下沉第一个数]	heap[1] = heap[size]; size--; down[1];
4.[删除任意一个元素][将第k个数等于最后一个数，删掉最后一个数，再上下这个数]	heap[k] = heap[size]; size--; down[k]; up[k];

操作	实现
5.修改任意一个元素	heap[k] = x; down[k]; up[k];

```

1  int h[100010],cnt;
2  void down(int u) {
3      int t = u;
4      if (u * 2 <= cnt && h[u * 2] < h[t]) t = u * 2;
5      if (u * 2 + 1 <= cnt && h[u * 2 + 1] < h[t]) t = u * 2 + 1; //令t为u及其子节点
    中的最小节点
6      if (u != t) { //判断u是不是最小节点，不是则与最小节点t交换，继续下沉(递归处理)
7          swap(h[u], h[t]);
8          down(t);
9      }
10 }

```

```

1  void up(int u) {
2      if (u / 2 && h[u / 2] > h[u]) { //诺比父节点小，则二者交换
3          swap(h[u / 2], h[u]);
4          u /= 2;
5      }
6  }

```

146. 序列

给定 m 个序列，每个包含 n 个非负整数。

现在我们可以从每个序列中选择一个数字以形成具有 m 个整数的序列。

很明显，我们一共可以得到 n^m 个这种序列，然后我们可以计算每个序列中的数字之和，并得到 n^m 个值。

求出这些序列和之中最小的 n 个值。

思路：第一个数组 $a[]$ 依次与其他数组合并，每次将 $a[]$ 更新最小的 n 个数

```

1  #include <iostream>
2  #include <algorithm>
3  #include <queue>
4  using namespace std;
5  using pii = pair<int,pair<int,int>>;
6  const int N = 2003;
7  int n,m;
8  int a[N],b[N],c[N];
9
10 struct Edge{
11     int s,x,y; //s为a[x]+b[y]的值
12     bool operator < (const Edge &e) const {
13         return s > e.s; //运算符重载，小根堆
14     }
15 };

```

```

16
17 void merge(){//数组a[]与数组b[]合并
18     priority_queue<Edge>pq;//a[]为有序数组, a[x],b[y]为n*n的组合, y一定时a[x]+b[y]有
    序
19     for(int i = 1;i <= n;i++){
20         pq.push({a[1]+b[i],1,i});
21     }
22     vector<int>ans;
23     for(int i = 1;i <= n;i++){
24         auto [s,x,y] = pq.top();//a[x]+b[y]出队, a[x+1]+b[y]入队
25         pq.pop();
26         c[i] = s;
27         x++;
28         pq.push({a[x]+b[y],x,y});
29     }
30     for(int i = 1;i <= n;i++){
31         a[i] = c[i];
32     }
33 }
34
35 void sol(){
36     cin >> m >> n;
37     for(int i = 1;i <= n;i++){ cin >> a[i]; }
38     sort(a+1,a+n+1);
39
40     m--;
41     while(m--){
42         for(int i = 1;i <= n;i++){ cin >> b[i]; }
43         merge();
44     }
45     for(int i = 1;i <= n;i++){ cout << a[i] << ' '; }
46     cout << endl;
47 }
48
49 int main(){
50     int T;cin >> T;
51     while(T--){ sol(); }
52 }

```

左偏树

左偏树(leftist tree)是一种可并堆, 具有堆的性质, 可以快速合并, 支持可持久化。

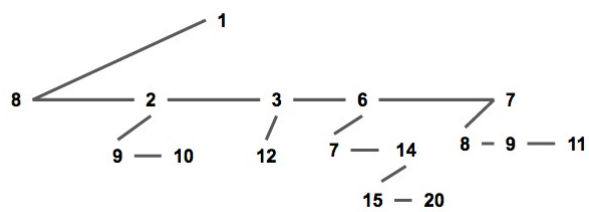
插入	查询最小值	删除最小值	合并
$O(\log N)$	$O(1)$	$O(\log N)$	$O(\log N)$

配对堆

配对堆是一个支持插入，查询/删除最小值，合并，修改元素等操作的数据结构，是一种**可并堆**。有速度快和结构简单的优势，但由于其为基于势能分析的均摊复杂度，不支持可持久化。

插入	查询最小值	删除最小值	合并
$O(1)$	$O(1)$	$O(\log N)$	$O(1)$

配对堆是一棵满足堆性质的带权多叉树，即每个节点的权值都小于或等于他的所有儿子。每个节点储存第一个儿子的指针，即链表的头节点；和他的右兄弟的指针。



查询最小值：根节点即为最小值。

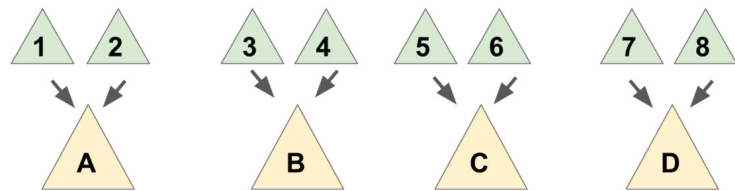
合并：令两个根节点较小的作为一个新的根节点，然后将较大的作为它的儿子插进去。

插入：新建一个节点然后与原堆合并即可。

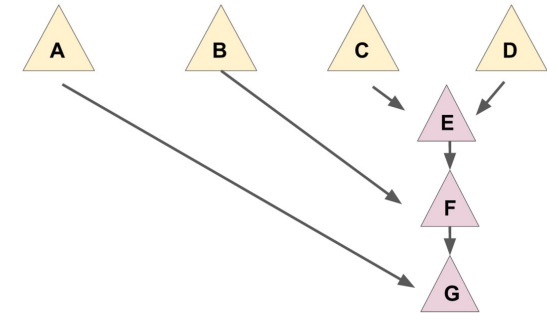
删除：删除操作较为麻烦，为了保证总的均摊复杂度，需要使用一个两步走的合并方法

1. 把儿子们两两配成一对，在把配成一对的儿子合并到一起。
2. 将新产生的堆从右往左（即老儿子到新儿子的方向）挨个合并到一起。

PASS #1



PASS #2



1 //封装实现,堆顶为最小值,未经过严谨测试

```

2  template<typename type>
3  class heap{
4      size_t heap_size;
5      struct node{
6          node *s,*t;//s指向该节点的第一个儿子,t指向该节点的下一个兄弟
7          type val;
8      }*root;
9
10     node *merge_bros(node *p){//辅助函数,合并一个节点的所有兄弟
11         if(p==nullptr || p->t==nullptr) return p;//如果该节点为空或它没有下一个兄弟,
    就不需要合并了
12         node *q=p->t;node *d=q->t;p->t=q->t=nullptr;
13         return merge(merge_bros(d),merge(p,q));
14     }
15
16     inline node *merge(node *p,node *q){//合并两个节点
17         if(p==nullptr) return q;
18         if(q==nullptr) return p;
19         if(p->val > q->val) swap(p,q);
20         q->t=p->s,p->s=q;
21         return p;
22     }
23
24     public:
25     inline heap<type>(){root=nullptr;}
26
27     inline void merge(node *p){
28         if(root==nullptr){root=p;return;}
29         if(p==nullptr) return;
30         if(root->val > p->val) swap(p,root);
31         p->t=root->s,root->s=p;
32     }
33
34     inline void merge(heap x){//合并两个堆,将堆x并入当前堆(不会清空x)
35         node *p=x.root;
36         heap_size += x.size();
37         if(root==nullptr){root=p;return;}
38         if(p==nullptr) return;
39         if(root->val > p->val) swap(p,root);
40         p->t=root->s,root->s=p;
41     }
42
43     inline void push(type x){
44         heap_size++;
45         node *p=new node;
46         p->val=x;p->s=nullptr,p->t=nullptr;
47         merge(p);
48     }
49
50     inline void pop(){
51         heap_size--;
52         node *T=root;

```

```

53     root=merge_bros(root->s);
54     delete T;
55 }
56
57 inline type top(){return root->val;}
58 inline bool empty(){return !heap_size;}
59 inline size_t size(){return heap_size;}
60 inline void clear(){root=nullptr;}
61 };
62
63 int main(){
64     heap<int> pq1,pq2;
65     pq1.push(2);pq2.push(1);
66     pq1.merge(pq2);
67     std::cout << pq1.top() << ' ' << pq1.size() << '\n';
68 }

```

哈希

开放寻址法

```

1  #include <iostream>
2  using namespace std;
3  const int N = 2000003, null = 0x3f3f3f3f; //N取大于所给范围两~三倍的一个质数，null取一个
    取不到的数
4  int h[N];
5  int find(int x) {
6      int k = (x % N + N) % N;
7      //如果x存在则返回x在哈希表中的位置，否则返回x应该在哈希表中插入的位置
8      while (h[k] != null && h[k] != x) { //如果这个位置有值且不是x，则k++，直到找到一个空
    位(null)
9          k++;
10         if (k == N) k = 0; //若k为N，则重新从0开始找
11     }
12     return k;
13 }
14 int main() {
15     memset(h, 0x3f, sizeof h); //初始化，memset按字节初始化，int类型只要填一个0x3f
16     int x; cin >> x;
17     int k = find(x);
18     //插入x
19     h[k] = x;
20
21     //查询x是否存在
22     if (h[k] == x) cout << "YES";
23     else cout << "NO";
24 }

```

```

25     return 0;;
26 }

```

拉链法

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4  const int N = 100003; //N取大于所给范围的第一个质数
5  int h[N], e[N], ne[N], idx; //e,ne为单链表用法, h为头结点
6
7  void insert(int x) {
8      int k = (x % N + N) % N; //k为哈希值
9      e[idx] = x;
10     ne[idx] = h[k];
11     h[k] = idx++;
12 }
13 bool find(int x) {
14     int k = (x % N + N) % N;
15     for (int i = h[k]; i != -1; i = ne[i]) {
16         if (e[i] == x) return true;
17     }
18     return false;
19 }
20
21 int main() {
22     memset(h, -1, sizeof h); //初始化为-1 (空指针一般用-1表示)
23
24 }

```

雪花匹配

定义Hash函数 $H(a_1 \dots a_6) = (\sum_{i=1}^6 a_i + \prod_{i=1}^6 a_i) \% mod$

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4  using ll = long long;
5  const int N = 100005, mod = 100003; //取接近N的质数
6  int h[N], ne[N], idx;
7  int snow[N][6]; //第idx个位置对应的雪花snow[idx][6]
8  int n;
9
10 int hs(int a[]) { //哈希函数
11     ll sum = 0, mul = 1;
12     for (int i = 0; i < 6; i++) {
13         sum = (sum + a[i]) % mod;

```

```

14     mul = mul*a[i]%mod;
15 }
16     return (sum+mul)%mod;
17 }
18
19 bool equal(int a[],int b[]){//判断两片雪花是否相等
20     for(int i = 0;i < 6;i++){
21         for(int j = 0;j < 6;j++){
22             bool eq = 1;
23             for(int k = 0;k < 6;k++){
24                 if(a[(i+k)%6] != b[(j+k)%6]) eq = 0;
25             }
26             if(eq) return 1;
27             eq = 1;
28             for(int k = 0;k < 6;k++){
29                 if(a[(i+k)%6] != b[(j-k+6)%6]) eq = 0;
30             }
31             if(eq) return 1;
32         }
33     }
34     return 0;
35 }
36
37 bool insert(int a[]){
38     int key = hs(a);
39
40     for(int i = h[key];i != -1;i = ne[i]){//诺发生哈希冲突
41         if(equal(snow[i],a)) return 1;//且出现两片雪花相等，返回1
42     }
43     //否则在哈希值对应链表位置接上该雪花
44     memcpy(snow[idx],a,6*sizeof(int));
45     ne[idx] = h[key];
46     h[key] = idx++;
47     return 0;
48 }
49
50 int main(){
51     cin >> n;
52     memset(h,-1,sizeof h);
53     for(int i = 1;i <= n;i++){
54         int a[10];
55         for(int j = 0;j < 6;j++) cin >> a[j];
56         if(insert(a)){
57             cout << "Twin snowflakes found.";
58             return 0;
59         }
60     }
61     cout << "No two snowflakes are alike.";
62 }

```

并查集

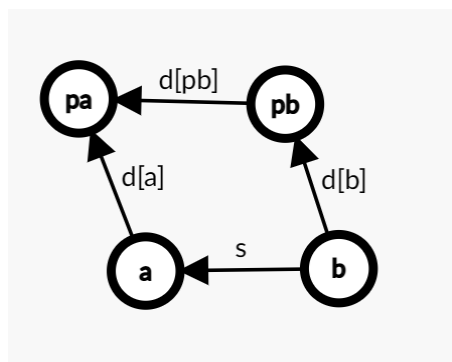
并查集(DSU)是一种用于管理元素所属集合的数据结构，实现为一个森林，其中每棵树表示一个集合，树中的节点表示对应集合中的元素

```
1  int p[N],siz[N];//p[i]存节点i的根节点，siz[i]存以i为根的集合大小
2
3  int find(int x){//查找x的根节点+路径压缩
4      if(p[x] != x) p[x] = find(p[x]);
5      return p[x];
6  }
7
8  void merge(int x,int y){
9      x = find(x), y = find(y);
10     if(x == y) return;
11     if(siz[x] > siz[y]) swap(x,y); //启发式合并,将节点少的连到节点多的,避免退化
12     siz[y] += siz[x];
13     p[x] = y;
14 }
15
16 int main(){
17     for(int i = 1;i <= n;i++) p[i] = i,siz[i] = 1;//初始化每个节点的父节点为自身
18 }
```

`i == p[i]` 父节点等于自身的数量即为连通块的数量[How Many Tables - HDU 1213 - Virtual Judge \(vjudge.net\)](https://vjudge.net/contest/428184#problem/D)

边带权并查集

并查集实际上是由若干棵树构成的森林，我们可以在树中的每条边上记录一个权值，即维护一个数组`d[]`，用`d[x]`保存节点`x`到父亲节点`p[x]`之间的边权。在每次路径压缩后，每个访问过的节点都会直接指向树根，如果我们同时更新这些节点的`d`值，就可以利用路径压缩过程来统计每个节点到树根之间的路径上的一些信息。



关系的传递可以看做向量。

如果 $pa == pb$ 那么我们可以检查 $d[a] + s$ 是否等于 $d[b]$ 来判断是否矛盾，或者求出相对关系大小 s 。

否则可以合并两者： $p[pb] = pa$, $d[pb] = d[a] + s - d[b]$ 。

[P1196 [NOI2002](https://www.luogu.com.cn/problem/P1196) 银河英雄传说 - 洛谷 (luogu.com.cn)]

有 N 只飞船， M 条指令，每条指令为以下之一：

1. $M\ i\ j$ ，表示让 i 号飞船所在的队列全部飞船保持原有顺序，接在第 j 号飞船所在队列的尾部
2. $C\ i\ j$ ，表示询问第 i 号飞船与第 j 号飞船当前是否处于同一列中，如果在，输出他们之间间隔了多少飞船

```
1  #include <iostream>
2
3  const int N = 30004;
4  int n = 30000, q;
5  int p[N], d[N], siz[N]; // d[x] 代表 x 与其父节点 (路径压缩后即根结点) 之间的边的权值，即位于 x 之前
   的飞船数量
6
7  int find(int x) { // 带权并查集模版，路径压缩时维护 d[]
8      if(p[x] != x) {
9          int root = find(p[x]);
10         d[x] += d[p[x]];
11         p[x] = root;
12     }
13     return p[x];
14 }
15
16 int main() {
17     for(int i = 1; i <= n; i++) p[i] = i, siz[i] = 1;
18     std::cin >> q;
19     while(q--){
20         char op; int a, b; std::cin >> op >> a >> b;
21         int pa = find(a), pb = find(b);
22         if(op == 'M'){
23             if(pa == pb) continue;
24             else{
25                 p[pa] = pb; // 将 a 所在队列并入 b 所在队列
26                 int s = siz[pb] - d[b] + d[a]; // s 即为 (b 到队列 pb 尾部的距离) + (队列 pa 头
   部到 a 的距离)
27                 d[pa] = d[b] + s - d[a];
28                 siz[pb] += siz[pa];
29             }
30         }
31         if(op == 'C'){
32             if(pa == pb) std::cout << std::max(0, std::abs(d[b] - d[a]) - 1) <<
   '\n';
33             else std::cout << -1 << '\n';
34         }
35     }
```

拓展域并查集

用于解决具有多个相互关系集合的问题。它是传统并查集的扩展，能够处理集合间的不同关系，如相互排斥或相互独立的关系。

[P1525 [NOIP 2010 提高组](#)] [关押罪犯 - 洛谷 \(luogu.com.cn\)](#)

n 个点有 m 对关系，把 n 个节点放入两个集合里，要求每对存在关系的两个节点不能放在同一个集合。问能否成功完成？

我们使用拓展域并查集，将 u 和 $u+n$ 分别表示 u 的两个相反状态，将 $(u, v+n)$ 连边表示 (u, v) 不应在一个集合里(也可以看作 u 和 v 的反状态在一个集合里)。相应的 $(v, u+n)$ 也应该连边。若 $(u, u+n)$ 在同一个集合里则说明发生了矛盾。

```

1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4  const int N = 40004, M = 100005;
5  int n, m;
6  int p[N];
7
8  struct node{
9      int a, b, c;
10     bool operator < (const node &e2) const{
11         return c > e2.c;
12     }
13 }e[M];
14
15 int find(int x){
16     if(p[x] != x) p[x] = find(p[x]);
17     return p[x];
18 }
19
20 void merge(int x, int y){
21     int px = find(x), py = find(y);
22     if(px != py) p[px] = py;
23 }
24
25 int main(){
26     cin >> n >> m;
27     for(int i = 1; i <= n << 1; i++) p[i] = i;
28
29     for(int i = 1; i <= m; i++){
30         cin >> e[i].a >> e[i].b >> e[i].c;
31     }
32     sort(e+1, e+m+1);

```

```

33
34     for(int i = 1;i <= m;i++){
35         auto [a,b,c] = e[i];
36         merge(a,b+n); merge(a+n,b);
37         if(find(a) == find(a+n) || find(b) == find(b+n)) { //发生矛盾
38             cout << c;
39             return 0;
40         }
41     }
42     cout << 0;
43 }

```

[P2024 [NOI2001](https://www.luogu.com.cn/problem/P2024)] 食物链 - 洛谷 ([luogu.com.cn](https://www.luogu.com.cn))

每个动物都是A, B, C的一种, 三者形成环形食物链, 依次给出m个句话, 为以下两种形式之一

1 x y 表示 x 和 y 是同类

2 x y 表示 x 吃 y

若当前话与前面的话产生矛盾, 则当前话为假话。求有多少假话。

我们用(x)表示同类域, (x+n)表示捕食域, (x+2n)表示天敌域

```

1  #include <iostream>
2
3  const int N = 50004*3;
4  int n,q;
5  int p[N],d[N];
6
7  int find(int x){
8      if(p[x] != x) {
9          int root = find(p[x]);
10         d[x] += d[p[x]];
11         p[x] = root;
12     }
13     return p[x];
14 }
15
16 void merge(int x,int y){
17     int px = find(x),py = find(y);
18     if(px == py) return;
19     p[px] = py;
20 }
21
22 int main(){
23     int ans = 0;
24     std::cin >> n >> q;
25     for(int i = 1;i <= n*3;i++) p[i] = i;
26     while(q--){
27         int op,x,y;std::cin >> op >> x >> y;
28         if(x > n || y > n) {ans++;continue;}
29         if(op == 1){ //x与y是同类

```

```

30         if(find(x) == find(y+n) || find(x) == find(y+n+n)) ans++;//如果y是x的
食物或y是x的天敌,矛盾
31         else{
32             merge(x,y);
33             merge(x+n,y+n);
34             merge(x+n+n,y+n+n);
35         }
36     }
37     else{//x吃y
38         if(find(x) == find(y+n+n) || find(x) == find(y)) ans++;//如果y是x的天
敌或y是x的同类,矛盾
39         else{
40             merge(x,y+n);
41             merge(x+n,y+n+n);
42             merge(x+n+n,y);
43         }
44     }
45 }
46 std::cout << ans;
47 }

```

可撤销并查集

[U531226【模板】可撤销并查集 - 洛谷](#)

维护 n 个集合，第 i 个集合初始只有一个元素 i 。执行 q 次操作：

1 x y：把元素 x 所在集合和元素 y 所在的集合合并。若已经在同一集合则跳过，否则记为一次有效合并。

2 x y：查询元素 x ， y 是否在同一集合

3：撤销上一次有效合并操作。

数据范围： $1 \leq n \leq 5 \times 10^5, 1 \leq q \leq 2 \times 10^6$

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct DSU {
5      int n;
6      vector<int> p, siz, stk; // stk用来记录每次有效合并中被合并的那个集合的根
7
8      DSU(int _n): n(_n) {
9          p.resize(n + 1);
10         siz.assign(n + 1, 1);
11         for (int i = 0; i <= n; i++) p[i] = i;

```

```

12     }
13
14     int find(int x) { // 不用路径压缩,保留原树结构,方便回溯
15         while (p[x] != x) x = p[x]; // 循环版,防止爆栈
16         return x;
17     }
18
19     void merge(int x, int y) {
20         x = find(x), y = find(y);
21         if (x == y) return;
22         if (siz[x] > siz[y]) swap(x, y);
23         stk.push_back(x); // 记录被合并的根x
24         p[x] = y;
25         siz[y] += siz[x];
26     }
27
28     bool same(int x, int y) {
29         return find(x) == find(y);
30     }
31
32     void undo(int t = 1) { // 撤销t次有效合并操作
33         while (t--) {
34             if (stk.empty()) return; // 如果没有有效操作了,直接返回
35             int x = stk.back(); // 取栈顶元素x,即被合并的根
36             stk.pop_back();
37             siz[p[x]] -= siz[x]; // 分离x并独立
38             p[x] = x;
39         }
40     }
41 };
42
43 void soviet() {
44     int n, q; std::cin >> n >> q;
45     DSU d(n);
46     while (q--) {
47         int op; std::cin >> op;
48         if (op == 1) {
49             int x, y; std::cin >> x >> y;
50             d.merge(x, y);
51         }
52         if (op == 2) {
53             int x, y; std::cin >> x >> y;
54             std::cout << (d.same(x, y) ? "Yes": "No") << '\n';
55         }
56         if (op == 3) {
57             d.undo();
58         }
59     }
60 }
61
62 int main() {
63     int M_T = 1; std::ios::sync_with_stdio(false); std::cin.tie(nullptr);

```

```

64 // std::cin >> M_T;
65 while (M_T--) { soviet(); }
66 return 0;
67 }

```

带删除并查集

删点并查集

[Problem - 2473. Junk-Mail Filter](#)

维护 n 个集合，第 i 个集合初始只有一个元素 i 。执行 q 次操作：

M x y：合并元素 x 所在集合和元素 y 所在集合。

S x：将元素 x 从集合中独立出来（后续可能还会被合并到其它集合）。

求最后有多少个集合。数据范围： $1 \leq n \leq 10^5, 1 \leq q \leq 10^6$ 。

使用虚拟节点，将每个真实节点映射到一个虚拟节点，删除时只需为其分配一个新的虚拟节点，原集合不受影响。

时间复杂度 $O(n + q)$

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct DSU {
5      int n;
6      vector<int> p, siz;
7
8      DSU(int _n): n(_n) {
9          p.resize(n + 1);
10         siz.resize(n + 1, 1);
11         for (int i = 0; i <= n; i++) p[i] = i;
12     }
13
14     int find(int x) {
15         if (p[x] != x) p[x] = find(p[x]);
16         return p[x];
17     }
18
19     void merge(int x, int y) {
20         x = find(x), y = find(y);
21         if (x == y) return;
22         if (siz[x] > siz[y]) swap(x, y);
23         siz[y] += siz[x];
24         p[x] = y;
25     }
26

```

```

27     bool same(int x, int y) {
28         return find(x) == find(y);
29     }
30 };
31
32 int n, q;
33 void soviet() { //本题 0_idx
34     DSU d(n + q); // 并查集大小开到 n + q
35     std::vector<int> actual(n);
36     for (int i = 0; i < n; i++) actual[i] = i; //actual[i]表示节点i当前对应的虚拟节
        点编号
37
38     int tot = n; //已分配的虚拟节点数目
39
40     while (q--) {
41         char op; std::cin >> op;
42         if (op == 'M') {
43             int x, y; std::cin >> x >> y;
44             d.merge(actual[x], actual[y]);
45         }
46         if (op == 'S') {
47             int x; std::cin >> x;
48             actual[x] = tot++; //为x分配一个新的虚拟节点,使其成为孤立节点
49         }
50     }
51
52     set<int> se; //这里也可以用桶优化
53     for (int i = 0; i < n; i++) { //统计所有真实节点所在集合
54         se.insert(d.find(actual[i]));
55     }
56     cout << se.size() << '\n';
57 }
58
59 int main() {
60     int M_T = 1; std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
61     // std::cin >> M_T;
62     int pid = 0;
63     while (std::cin >> n >> q, n) {
64         std::cout << "Case #" << ++pid << ": ";
65         soviet();
66     }
67     return 0;
68 }

```

删边并查集

方案一：考虑Link-Cut Tree维护动态森林，支持在线操作，代码量较高

方案二：可撤销并查集+线段树分治，只能离线

可持久化并查集

[P3402 【模板】可持久化并查集 - 洛谷](#)

维护 n 个集合，每个集合 i 初始只有元素 i 。

1 $x\ y$ ：合并元素 x 所在集合和元素 y 所在集合。

2 j ：回到第 k 次操作（三次操作任意一种都算一次操作），若 $k=0$ 表示回到初始状态。

3 $x\ y$ ：询问元素 x ， y 是否处于同一集合。

数据范围： $1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$

采用可持久化线段树，作为可持久化数组，每个版本对应一颗线段树的根。线段树的每个叶子节点存储父节点编号和深度。

查询操作，时间复杂度 $O(\log^2 n)$

合并操作，时间复杂度 $O(\log^2 n)$ ，单次新增空间 $O(\log n)$

回溯操作， $O(1)$ ，无额外开销

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int N = 100005;
5  int n, m;
6
7  struct ST {
8      int l, r;
9      int fa, dep;
10 } t[N * 40]; // 节点池：初始建树4*N + 最多有效合并2 * [log2 N] * N
11
12 int tot; // 当前节点数
13
14 int build(int l, int r) {
15     int p = ++tot;
16     if (l == r) { // 初始时每个节点的根为自己,深度为1
17         t[p].fa = l;
18         t[p].dep = 1;
19         return p;
20     }
21     int mid = (l + r) >> 1;
22     t[p].l = build(l, mid);
23     t[p].r = build(mid + 1, r);
24     return p;
25 }
26

```

```

27 int update(int rt, int l, int r, int pos, int fa, int dep) {
28     int p = ++tot; //在rt版本基础上,将位置pos的fa和dep修改为新值,只复制路径上的节点,其余节
    点共享
29     t[p] = t[rt];
30     if (l == r) {
31         t[p].fa = fa;
32         t[p].dep = dep;
33         return p;
34     }
35     int mid = (l + r) >> 1;
36     if (pos <= mid) t[p].l = update(t[p].l, l, mid, pos, fa, dep);
37     else t[p].r = update(t[p].r, mid + 1, r, pos, fa, dep);
38     return p;
39 }
40
41 pair<int, int> query(int p, int l, int r, int pos) { //在版本p中查询位置pos的
    (fa,dep)
42     if (l == r) return {t[p].fa, t[p].dep};
43     int mid = (l + r) >> 1;
44     if (pos <= mid) return query(t[p].l, l, mid, pos);
45     else return query(t[p].r, mid + 1, r, pos);
46 }
47
48
49 // 在版本 rt 下查找 x 的根
50 int find(int rt, int x) { //不使用路径压缩
51     while (1) {
52         auto [fa, dep] = query(rt, 1, n, x);
53         if (fa == x) return x;
54         x = fa;
55     }
56 }
57
58 // 在版本 rt 上合并 x,y, 返回新版本根
59 int merge(int rt, int x, int y) {
60     x = find(rt, x), y = find(rt, y);
61     if (x == y) return rt;
62
63     auto [vx, dx] = query(rt, 1, n, x); // 获取两个根的深度
64     auto [vy, dy] = query(rt, 1, n, y);
65
66     if (dx > dy) swap(x, y), swap(dx, dy); // 按深度合并, 将x合并到y
67     int new_rt = update(rt, 1, n, x, y, dx); //x的fa修改为y,dep不变
68     if (dx == dy) new_rt = update(new_rt, 1, n, y, y, dy + 1); //若合并前x和y的深度
    相等,则合并后将y的深度+1
69     return new_rt; // 返回新版本的根
70 }
71
72 bool same(int rt, int x, int y) {
73     return find(rt, x) == find(rt, y);
74 }
75

```

```

76 int main() {
77     cin >> n >> m;
78     vector<int> ver(m + 1); // 版本号, ver[0] 为初始状态
79     ver[0] = build(1, n);
80
81     for (int i = 1; i <= m; ++i) {
82         int op; cin >> op;
83         if (op == 1) {
84             int x, y; cin >> x >> y;
85             ver[i] = merge(ver[i - 1], x, y);
86         }
87         if (op == 2) {
88             int k; cin >> k;
89             ver[i] = ver[k];
90         }
91         if (op == 3) {
92             int x, y; cin >> x >> y;
93             ver[i] = ver[i - 1];
94             cout << same(ver[i], x, y) << '\n';
95         }
96     }
97 }

```

分块

数列分块

将原数列分成 $\sqrt{N}$ 个块，修改和查询的时候可以直接利用整块的信息，优化时间复杂度。

[分块思想 - OI Wiki \(oi-wiki.org\)](https://oi-wiki.org/)

[例题：分块入门 \(loj.ac\)](https://loj.ac/)

$O(N)$ 预处理， $O(\sqrt{N})$ 修改/查询，支持区间修改，单点插入，区间查询等操作，较为灵活

元素个数n (1~n)	
每一块的最长长度m	$m = \text{sqrt}(n)$
块的个数num	$\text{num} = (n/m) + \text{bool}(n\%m)$
第i个元素所在的块id[i]	$\text{id}[i] = (i-1)/m + 1$
第i个元素所在块的左端点b[id[i]].l	$\text{if}(\text{b}[\text{id}[i]].l == 0) \text{b}[\text{id}[i]].l = i$

元素个数n (1~n)	
第i个元素所在块的右端点b[id[i]].r	b[id[i]].r = max(b[id[i]].r , i)

```

1 //数据定义
2 int id[N]; //第i个元素在哪个块中
3 struct Block{ //存每一块的属性
4     int l,r; //该块的左右端点
5     ll lazy; //lazy标记
6     ll sum; //区间和
7     ll add; //区间加标记
8     ll mul; //区间乘标记
9     ll cnt; //块中元素个数标记
10    //...
11 }b[N];
12
13 vector<int>v[N]; //如果需要对块进行排序、拷贝等操作，可以开vector数组将每一个元素放入对应块中

```

```

1 //初始化
2 void initi(){
3     m = sqrt(n);
4     for(int i = 1; i <= n; i++){
5         id[i] = (i-1)/m + 1; //第i个元素在第id[i]块
6         if(b[id[i]].l == 0) b[id[i]].l = i; //第id[i]块的左右端点
7         b[id[i]].r = max(b[id[i]].r, (ll)i);
8         //v[id[i]].emplace_back(a[i]);
9     }
10 }

```

```

1 //修改/查询
2
3 //如果l和r在同一块
4 if(id[l] == id[r]){
5     for(int i = l; i <= r; i++){
6         //直接枚举l~r所有元素
7     }
8 }
9
10 //如果l和r不在同一块
11 else{
12     for(int i = l; id[i] == id[l]; i++){
13         //枚举l所在块
14     }
15     for(int i = r; id[i] == id[r]; i--){
16         //枚举r所在块

```

```

17     }
18     for(int i = id[l]+1;i < id[r];i++){
19         //l~r之间所有完整块利用区间属性快速操作
20     }
21 }

```

```

1 //区间修改，区间求小于x的数的个数: https://loj.ac/p/6278
2 //区间修改，区间求大于等于x的数的个数: https://www.luogu.com.cn/problem/P2801
3 #include <iostream>
4 #include <algorithm>
5 #include <cmath>
6 #include <vector>
7 using namespace std;
8 using ll = long long;
9 const ll N = 100005;
10 ll n,m,cnt;
11 ll a[N];
12
13 ll id[N];
14 vector<ll>v[N]; //v[i]存第i块排序后的序列
15 struct Bolck{
16     ll l,r,lazy;
17 }b[N];
18
19 void initi(){
20     m = sqrt(n);
21     for(int i = 1;i <= n;i++){
22         id[i] = (i-1)/m + 1;
23         v[id[i]].emplace_back(a[i]);
24         if(b[id[i]].l == 0) b[id[i]].l = i;
25         b[id[i]].r = max(b[id[i]].r,(ll)i);
26     }
27     for(int i = 1;i <= id[n];i++){
28         sort(v[i].begin(),v[i].end());
29     }
30 }
31
32 void bsort(int i){ //块排序
33     v[i].clear();
34     for(int j = b[i].l;j <= b[i].r;j++){
35         v[i].emplace_back(a[j]);
36     }
37     sort(v[i].begin(),v[i].end());
38 }
39
40 void add(ll l,ll r,ll c){
41     if(id[l] == id[r]){
42         for(int i = l;i <= r;i++){

```

```

43         a[i]+=c;
44     }
45     bsort(id[l]);
46 }
47 else{
48     for(int i = l;id[i] == id[l];i++){
49         a[i]+=c;
50     }
51     for(int i = r;id[i] == id[r];i--){
52         a[i]+=c;
53     }
54     for(int i = id[l]+1;i < id[r];i++){
55         b[i].lazy += c;
56     }
57     bsort(id[l]);
58     bsort(id[r]);
59 }
60 }
61
62 ll query(int l,int r,int c){
63     ll ans = 0;
64     if(id[l] == id[r]){
65         for(int i = l;i <= r;i++){
66             ans+=(a[i] < c-b[id[i]].lazy);
67         }
68     }
69     else{
70         for(int i = l;id[i] == id[l];i++){
71             ans+=(a[i] < c-b[id[i]].lazy);
72         }
73         for(int i = r;id[i] == id[r];i--){
74             ans+=(a[i] < c-b[id[i]].lazy);
75         }
76         for(int i = id[l]+1;i < id[r];i++){//l~r之间每个完整块v[i]二分查找
77             ans+=lower_bound(v[i].begin(),v[i].end(),c-b[i].lazy)-v[i].begin();
78         }
79     }
80     return ans;
81 }
82
83 int main(){
84     cin >> n;
85     for(int i = 1;i <= n;i++){ cin >> a[i]; }
86     initi();
87     for(int i = 1;i <= n;i++){
88         ll op,l,r,c;cin >> op >> l >> r >> c;
89         if(op == 0){ add(l,r,c);}
90         else{ cout << query(l,r,c*c) <<endl; }
91     }
92 }

```

```

1 //单点插入, 单点查询
2 #include <iostream>
3 #include <vector>
4 #include <cmath>
5 using namespace std;
6 const int N = 100005<<1;
7 int n,m,all,num;//n: 初始元素数量, all: 插入后元素数量, m: 块长度, num: 块数量
8 int a[N];
9
10 vector<int>v[N];
11 int id[N];
12
13 void initi(){
14     m = sqrt(all);
15     num = all/m + bool (all%m);
16     for(int i = 1;i <= all;i++){
17         id[i] = (i-1)/m + 1;
18         v[id[i]].emplace_back(a[i]);
19     }
20 }
21
22 void reint(){//重新分块,保证每块都接近sqrt(all),否则可能退化为普通数组
23     int cnt = 1;
24     for(int i = 1;i <= num;i++){
25         for(int j = 0;j < v[i].size();j++){
26             a[cnt++] = v[i][j];
27         }
28         v[i].clear();
29     }
30 }
31
32 void ins(int l,int x){
33     int cnt = 1;
34     while(l > v[cnt].size()){//根据每个块的长度推出第l个元素的位置
35         l -= v[cnt].size();
36         cnt++;
37     }
38     v[cnt].insert(v[cnt].begin()+ l-1,x);
39     all++;
40
41     if(v[id[l]].size() > 2*m){//如果插入后, 块的长度>2*m,则清空v[]还原到a[],重新初始化
        分块
42         reint();
43         initi();
44     }
45 }
46
47 int query(int r){
48     int cnt = 1;
49     while(r > v[cnt].size()){
50         r -= v[cnt].size();
51         cnt++;

```

```

52     }
53     return v[cnt][r-1];
54 }
55
56 int main(){
57     cin >> n;
58     all = n;
59     for(int i = 1; i <= n; i++){
60         cin >> a[i];
61     }
62     init1();
63     for(int i = 1; i <= n; i++){
64         int op, l, r, c; cin >> op >> l >> r >> c;
65         if(op == 0){ ins(l, r); }
66         else{ cout << query(r) << endl; }
67     }
68 }

```

值域分块

值域分块维护的是一个集合，类似桶排，常配合离散化使用。可以做到 $O(1)$ 插入、删除一个数， $O(\sqrt{N})$ 查询全局排名、全局第 k 大、前驱、后继等操作。在插入频繁而查询较少时，可以代替甚至优于平衡树(查询/修改都是 $\log$ 级别)，比如莫队。

[P2617 Dynamic Rankings - 洛谷](#)

单点修改，求区间第 k 小， $N, Q \leq 10^5$

这里使用带修莫队 + 值域分块解决。

`cnt[x]` 记录数字 x 出现的次数

`cnt2[block]` 记录第 `block` 块内所有数字出现次数总和

修改时同时让 `cnt[x]++`, `cnt2[blo[x]]++`;

查询第 k 小时，从第一块开始遍历，直到当前块的前缀和大于 k ，再遍历这一块找到第一个数 j 的前缀大于 k

```

1  #include <iostream>
2  #include <vector>
3  #include <cmath>
4  #include <algorithm>
5
6  const int N = 200005;
7  int n, q, siz;
8  int a[N];
9  std::vector<int> hs(1, -2e9);
10

```

```

11 struct Ask {
12     int l, r, t, id, k;
13     bool operator < (const Ask &e2) const {
14         if ((l - 1) / siz != (e2.l - 1) / siz) return l < e2.l;
15         if ((r - 1) / siz != (e2.r - 1) / siz) return r < e2.r;
16         return t < e2.t;
17     }
18 }ak[N];
19
20 struct Update {
21     int pos, val;
22 }update[N];
23
24 int qid, uid;
25 int ans[N];
26 int cnt[N], cnt2[N], blo[N];
27
28 void add(int x) {
29     cnt[x]++;
30     cnt2[blo[x]]++;
31 }
32
33 void del(int x) {
34     cnt[x]--;
35     cnt2[blo[x]]--;
36 }
37
38 void mov(int l, int r, int x) {
39     auto &[pos, val] = update[x];
40     if (l <= pos && r >= pos) {
41         del(a[pos]);
42         add(val);
43     }
44     std::swap(val, a[pos]);
45 }
46
47 int get_kth(int k) { //查询第k小
48     int res = 0, now = 1;
49     while (res + cnt2[now] < k) res += cnt2[now++];
50     for (int j = (now - 1) * siz + 1; ;j++) {
51         res += cnt[j];
52         if (res >= k) return j;
53     }
54     return 0;
55 }
56
57 void init() {
58     siz = std::pow(n, 2.0 / 3);
59     std::sort(ak + 1, ak + qid + 1);
60
61     for (int i = 1; i < hs.size(); i++) {
62         blo[i] = (i - 1) / siz + 1;

```

```

63     }
64
65     for (int i = 1, l = 1, r = 0, t = 0; i <= qid; i++) {
66         while (l > ak[i].l) add(a[--l]);
67         while (r < ak[i].r) add(a[++r]);
68         while (l < ak[i].l) del(a[l++]);
69         while (r > ak[i].r) del(a[r--]);
70         while (t < ak[i].t) mov(ak[i].l, ak[i].r, ++t);
71         while (t > ak[i].t) mov(ak[i].l, ak[i].r, t--);
72         ans[ak[i].id] = get_kth(ak[i].k);
73     }
74 }
75
76 int main() {
77     std::ios::sync_with_stdio(false); std::cin.tie(0);
78     std::cin >> n >> q;
79     for (int i = 1; i <= n; i++) {
80         std::cin >> a[i];
81         hs.emplace_back(a[i]);
82     }
83
84     for (int i = 1; i <= q; i++) {
85         char op; std::cin >> op;
86         if (op == 'Q') {
87             int l, r, k; std::cin >> l >> r >> k;
88             ++qid;
89             ak[qid] = {l, r, uid, qid, k};
90         }
91         if (op == 'C') {
92             int pos, val; std::cin >> pos >> val;
93             hs.emplace_back(val);
94             ++uid;
95             update[uid] = {pos, val};
96         }
97     }
98
99     std::sort(hs.begin() + 1, hs.end());
100    hs.erase(std::unique(hs.begin() + 1, hs.end()), hs.end());
101
102    auto get = [&](int x) -> int {
103        return std::lower_bound(hs.begin() + 1, hs.end(), x) - hs.begin();
104    };
105
106    for (int i = 1; i <= n; i++) {
107        a[i] = get(a[i]);
108    }
109    for (int i = 1; i <= uid; i++) {
110        update[i].val = get(update[i].val);
111    }
112
113    init();
114

```

```

115     for (int i = 1; i <= qid; i++) {
116         std::cout << hs[ans[i]] << '\n';
117     }
118 }

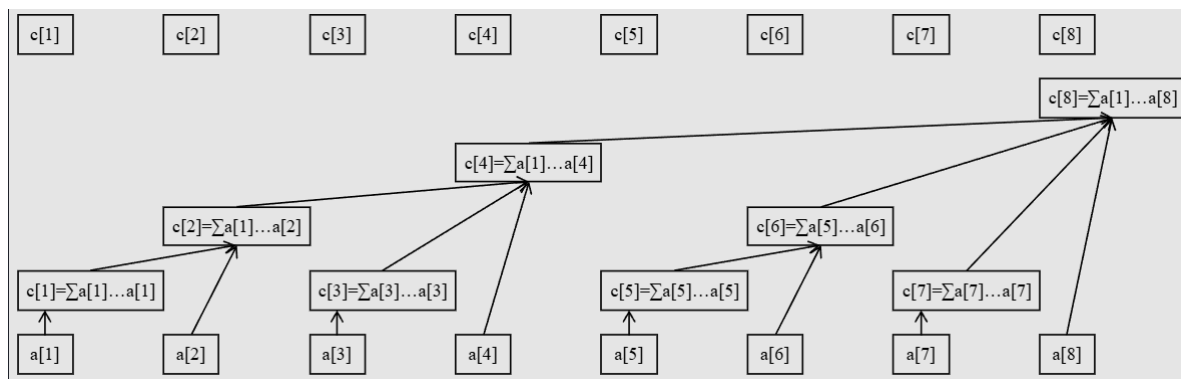
```

树状数组

树状数组是(Fenwick Tree)一种支持**单点修改**和**区间查询**，代码量小的数据结构，树状数组能解决的问题是线段树能解决的问题的子集

普通树状数组维护的信息及运算要满足 **结合律** 且 **可差分**，如加法、乘法、异或等。

一维树状数组



```

1  template<typename T>
2  struct Fenwick { //1_idx
3      int n;
4      std::vector<T> t;
5
6      explicit Fenwick(int _n = 0) : n(_n) {
7          t.assign(_n + 1, T{});
8      }
9
10     void add(int i, const T& x) {
11         while (i <= n) {
12             t[i] += x;
13             i += i & -i;
14         }
15     }
16
17     T sum(int i) {
18         T ans = 0;
19         while (i) {
20             ans += t[i];
21             i -= i & -i;

```

```

22     }
23     return ans;
24 }
25
26
27 int kth(T k) {
28     int x = 0;
29     T ans = 0;
30     for (int i = std::log2(n); i >= 0; i--) {
31         int nx = x + (1 << i);
32         if (nx <= n && ans + t[nx] < k) {
33             ans += t[nx];
34             x = nx;
35         }
36     }
37     return x + 1;
38 }
39
40 T sum(int l, int r) { return sum(r) - sum(l - 1); }
41 T query(int i) { return sum(i); }
42 T query(int l, int r) { return sum(l, r); }
43 };
44
45 Fenwick<int>t(n);

```

区间修改，单点查询

```

1 //利用差分及前缀和数组，t[]建树为差分数组
2 #include <iostream>
3 using namespace std;
4 using ll = long long;
5 const int N = 1000006;
6 int n,q;
7 ll a[N],t[N];
8
9 ll lowbit(ll x){return x&-x;}
10
11 void add(ll i,ll x){
12     while(i <= n){
13         t[i] += x;
14         i += lowbit(i);
15     }
16 }
17
18 ll getsum(int i){
19     ll ans = 0;
20     while(i > 0){
21         ans += t[i];
22         i -= lowbit(i);
23     }

```

```

24     return ans;
25 }
26
27 int main(){
28     cin >> n >> q;
29     for(int i = 1; i <= n; i++){
30         cin >> a[i];
31     }
32
33     while(q--){
34         int op; cin >> op;
35         if(op == 1){ //区间l~r加上x
36             int l, r, x; cin >> l >> r >> x;
37             add(l, x); add(r+1, -x);
38         }
39         else{ //查询下标为x的数所在的值
40             int x; cin >> x;
41             cout << a[x] + getsum(x) << '\n';
42         }
43     }
44 }

```

二维树状数组

- 子矩阵加上一个数
- 求子矩阵的和

```

1 //https://www.luogu.com.cn/problem/P4514
2 #include <iostream>
3 #include <vector>
4
5 template<typename T>
6 struct Fenwick{
7     int n, m;
8     std::vector<std::vector<T>> t1, t2, t3, t4;
9
10     Fenwick () {}
11     Fenwick (int _n, int _m) : n(_n), m(_m) {
12         t1 = t2 = t3 = t4 = std::vector<std::vector<T>> (n + 1, std::vector<T>(m
13 + 1));
14     }
15
16     T lowbit(T x) { return x & -x; }
17
18     void add(int x, int y, T z) {
19         for (int i = x; i <= n; i += lowbit(i)) {
20             for (int j = y; j <= m; j += lowbit(j)) {

```

```

20         t1[i][j] += z;
21         t2[i][j] += z * x; // 注意是 z * x 而不是 z * i, 后面同理
22         t3[i][j] += z * y;
23         t4[i][j] += z * x * y;
24     }
25 }
26 }
27
28 void range_add(int x1, int y1, int x2, int y2, T z) { //(x1, y1) 到 (x2, y2)
子矩阵
29     add(x1, y1, z);
30     add(x1, y2 + 1, -z);
31     add(x2 + 1, y1, -z);
32     add(x2 + 1, y2 + 1, z);
33 }
34
35 T sum(int x, int y) {
36     T res = 0;
37     for (int i = x; i; i -= lowbit(i)) {
38         for (int j = y; j; j -= lowbit(j)) {
39             res += (x + 1) * (y + 1) * t1[i][j] - (y + 1) * t2[i][j] - (x +
1) * t3[i][j] + t4[i][j];
40         }
41     }
42     return res;
43 }
44
45 T range_sum(int x1, int y1, int x2, int y2) {
46     return sum(x2, y2) - sum(x2, y1 - 1) - sum(x1 - 1, y2) + sum(x1 - 1, y1
- 1);
47 }
48 };
49
50 const int N = 305;
51 int a[N][N];
52
53 int main() {
54     char op; int n, m; std::cin >> op >> n >> m;
55     Fenwick<int> t(n, m);
56
57     while(std::cin >> op) {
58         if (op == 'L') {
59             int x1, y1, x2, y2, dx; std::cin >> x1 >> y1 >> x2 >> y2 >> dx;
60             t.range_add(x1, y1, x2, y2, dx);
61         }
62         else {
63             int x1, y1, x2, y2; std::cin >> x1 >> y1 >> x2 >> y2;
64             std::cout << t.range_sum(x1, y1, x2, y2) << '\n';
65         }
66     }
67 }

```

权值树状数组

若题目允许离线，则离散化后空间复杂度为 $O(N)$ ，否则需要开到 $O(\text{值域})$ 的大小。

全局第k小

```
1 // 权值树状数组查询全局第 k 小，倍增代替二分， $O(\log N)$ 
2 int kth(int k){//返回第一个 前缀和 $\geq k$  的下标,不存在返回n+1
3     int sum = 0,x = 0;
4     for(int i = log2(n);i >= 0;i--){//要注意n应为b[]的值域而非数组大小
5         int nx = x + (1 << i);
6         if(nx <= n && sum + t[nx] < k){//诺成功扩展
7             x = nx;
8             sum += t[x];
9         }
10    }
11    return x + 1; //最后一个<k的位置 + 1
12 }
```

[P10688 Buy Tickets - 洛谷](#)

给定 n 次操作 (p,v) ，每次在第 p 个位置后面插入值 v ，输出最后形成的队列

与[谜一样的牛]那题类似，我们先把所有位置填满空位，然后倒序处理。每次查找第 $p+1$ 个空位，放置 v ，然后将位置 $p+1$ 标记为非空。

```
1 #include <cstdio>
2 #include <cmath>
3
4 const int N = 200005;
5 int n,v[N],p[N],ans[N];
6
7 int t[N];
8 void add(int i,int x){
9     while(i <= n) {
10         t[i] += x;
11         i += i&-i;
12     }
13 }
14
15 int kth(int k){
16     int sum = 0,x = 0;
17     for(int i = std::log2(n);i >= 0;i--){
18         int nx = x + (1 << i);
19         if(nx <= n && sum + t[nx] < k) {
```

```

20         x = nx;
21         sum += t[nx];
22     }
23 }
24 return x+1;
25 }
26
27 void sol(){
28     for(int i = 1;i <= n;i++){
29         scanf("%d %d",&p[i],&v[i]);
30         add(i,1);
31     }
32     for(int i = n;i >= 1;i--){
33         int pos = kth(p[i]+1);
34         ans[pos] = v[i];
35         add(pos,-1);
36     }
37     for(int i = 1;i <= n;i++){
38         printf("%d ",ans[i]);
39     }
40     puts("");
41 }
42
43 int main(){
44     while(~scanf("%d",&n)) sol();
45 }

```

P3369 【模板】普通平衡树 - 洛谷 (luogu.com.cn)

您需要动态地维护一个可重集合 M ，并且提供以下操作：

1. 向 M 中插入一个数 x 。
2. 从 M 中删除一个数 x （若有多个相同的数，应只删除一个）。
3. 查询 M 中有多少个数比 x 小，并且将得到的答案加一。
4. 查询如果将 M 从小到大排列后，排名位于第 x 位的数。
5. 查询 M 中 x 的前驱（前驱定义为小于 x ，且最大的数）。
6. 查询 M 中 x 的后继（后继定义为大于 x ，且最小的数）。

对于操作 3,5,6，**不保证**当前可重集中存在数 x 。

本题中我们只需要关心的是每个数之间的相对关系，且允许离线，因此可以离散化处理。

```

1 #include <iostream>
2 #include <cmath>
3 #include <algorithm>
4 #include <vector>
5 using namespace std;
6 const int N = 100005;
7 int op[N],a[N];

```

```

8  int n;
9  vector<int>hs;
10
11  template<typename T>
12  struct Fenwick{
13      int n;
14      std::vector<T>t;
15
16      Fenwick(int _n = 0){
17          init(_n);
18      }
19
20      void init(int _n){
21          n = _n;
22          t.assign(n+1,T{});
23      }
24
25      void add(int i,const T &x){
26          while(i <= n){
27              t[i] += x;
28              i += i&-i;
29          }
30      }
31
32      T sum(int i){
33          T ans = 0;
34          while(i){
35              ans += t[i];
36              i -= i&-i;
37          }
38          return ans;
39      }
40
41      T sum(int l,int r){
42          return sum(r) - sum(l-1);
43      }
44
45      T kth(T k){
46          T sum = 0,x = 0;
47          for(int i = std::log2(n);i >= 0;i--){
48              T nx = x + (1 << i);
49              if(nx <= n && sum + t[nx] < k){
50                  x = nx;
51                  sum += t[x];
52              }
53          }
54          return x+1;
55      }
56
57      T query(int i) {return sum(i);}
58      T query(int l,int r) {return sum(l,r);}
59  };

```

```

60
61 int main(){
62     int q;cin >> q;
63     for(int i = 1;i <= q;i++){
64         cin >> op[i] >> a[i];
65         if(op[i] != 4) hs.emplace_back(a[i]);
66     }
67
68     hs.emplace_back(-1e9);
69     sort(hs.begin(),hs.end());
70     hs.erase(unique(hs.begin(),hs.end()),hs.end());
71     n = hs.size()-1;
72     for(int i = 1;i <= q;i++){
73         if(op[i] != 4) a[i] = lower_bound(hs.begin(),hs.end(),a[i])-hs.begin();
74     }
75
76     Fenwick<int>t(n+5);
77     for(int i = 1;i <= q;i++){
78         if(op[i] == 1) t.add(a[i],1);//插入一个数x
79         if(op[i] == 2) t.add(a[i],-1);//删除一个数x
80         if(op[i] == 3) cout << t.sum(a[i]-1)+1 << '\n';//a[i]的排名,即区间
[1,q[i]-1]的数的个数再加1
81         if(op[i] == 4) cout << hs[t.kth(a[i])] << '\n';//查询全局第k小的数(下标从1开
始)
82         if(op[i] == 5) cout << hs[t.kth(t.sum(a[i]-1))] << '\n';//前驱(若不存在则
返回x)
83         if(op[i] == 6) cout << hs[t.kth(t.sum(a[i])+1)] << '\n';//后继(若不存在则
返回x)
84     }
85 }

```

静态区间颜色数

给点一个长度为 n 的数组 $a[]$ ， q 个询问：区间 $[l,r]$ 不同元素的个数。

思路：对于询问多个区间 $[L,R]$ 中出现不同数字个数，当 R 相同时，如果区间出现了重复数字，那么我们只需要关心最右端的这个数字就可以了。换言之，重复出现的数字在左端将无任何贡献。例如 $\{1\ 2\ 3\ 1\ 4\}$ ，在询问区间 $[1,4]$ 时，第一个1无任何贡献，并且当询问的 R 不断右移的过程中，第一个1都无任何意义，被第四位置的替代掉。

于是把**询问离线**下来，按照询问的右端点排序。

当出现一个数字时，如果这个数字曾经出现，则 `add(1st[pos[i]],-1)`，把这个数在上一个位置的标记清掉，`add(pos[i],1)` 来更新这个数字的新位置。只需要维护第 i 个位置上有没有数，求区间 $[1,r]$ 内有多少个数。

用树状数组单点修改、区间查询。用前缀和 `query(R)-query(L-1)` 把区间 $[L,R]$ 的数的个数出来。

```

1 //HH的项链 https://www.luogu.com.cn/problem/P1972
2 #include <iostream>
3 #include <algorithm>
4 using namespace std;
5 const int N = 1000006;
6 int n,q;
7 int a[N];//本题a[i] <= 1e6,不用离散化
8 int pos[N];
9 int ans[N];
10
11 struct Ask{
12     int l,r,id;
13     bool operator < (Ask &e2){
14         return r < e2.r;
15     }
16 }ask[N];
17
18 int t[N];
19
20 void add(int i,int x){
21     while(i <= n){
22         t[i] += x;
23         i += i&-i;
24     }
25 }
26
27 int query(int i){
28     int ans = 0;
29     while(i > 0){
30         ans += t[i];
31         i -= i&-i;
32     }
33     return ans;
34 }
35
36 int main(){
37     cin >> n;
38     for(int i = 1;i <= n;i++) cin >> a[i];
39     cin >> q;
40     for(int i = 1;i <= q;i++){
41         cin >> ask[i].l >> ask[i].r;
42         ask[i].id = i;
43     }
44     sort(ask+1,ask+q+1);//按右端点升序排序
45
46     for(int i = 1,j = 1;i <= q;i++){//处理每个询问ask[i],j为当前右端点。
47         auto &[l,r,id] = ask[i];
48         while(j <= r){//更新j到ask[i].r的值
49             if(pos[a[j]]) add(pos[a[j]],-1);//若a[j]之前已经出现过则删去
50             pos[a[j]] = j;
51             add(j,1);
52             j++;

```

```

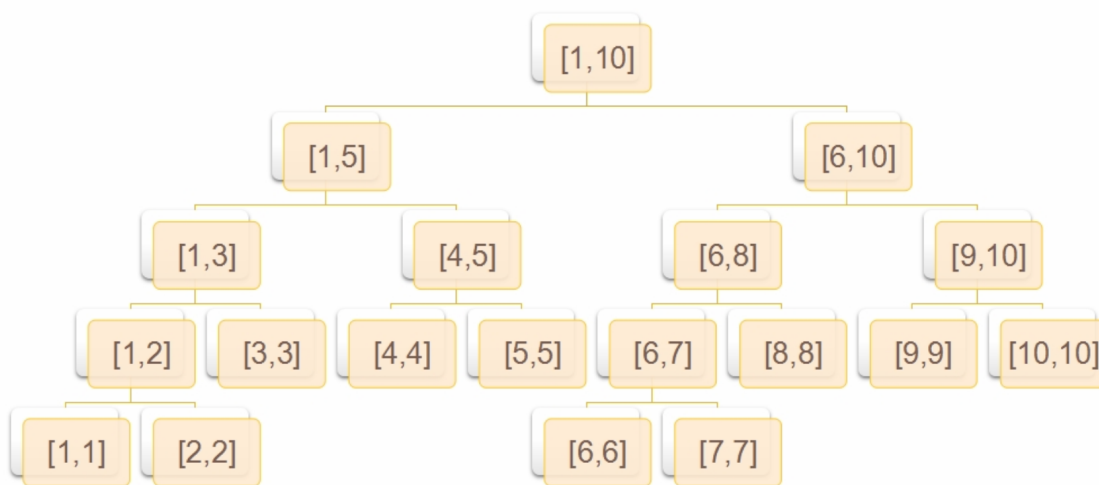
53     }
54     ans[id] = query(r) - query(l-1);
55 }
56 for(int i = 1; i <= q; i++) cout << ans[i] << '\n';
57 }

```

线段树

线段树是常用的用来维护 **区间信息** 的数据结构。

可以在 $O(\log N)$ 的时间复杂度内实现单点/区间修改、区间查询 (sum、max、min) 等操作。



建树

```

1  struct ST{
2      int l,r;
3      int dat;//lazy,mx,mn,sum,gcd...
4  }t[N<<2]; //至少要开 4*N 空间
5
6  void build(int p,int l,int r){ //build(1,1,n);
7      t[p] = {l,r,0};
8      if(l == r){t[p].dat = a[l];return;}
9      int mid = l + r >> 1;
10     build(p+p,l,mid);build(p+p+1,mid+1,r); //递归建树
11     //t[p].dat = f(t[p+p].dat,t[p+p+1].dat);
12 }

```

单点修改，区间查询

单点修改：从根节点开始遍历，递归找到需要修改的叶子节点，然后修改，然后向上传递信息。

区间查询：

- 1.若当前节点所表示的区间已经被询问区间所完全覆盖，则立即回溯，并传回该点的信息。
- 2.若当前节点的左儿子所表示的区间已经被询问区间所完全覆盖，就递归访问它的左儿子。
- 3.若当前节点的右儿子所表示的区间已经被询问区间所完全覆盖，就递归访问它的右儿子。

```
1 //单点修改，区间最值 http://acm.hdu.edu.cn/showproblem.php?pid=1754
2 void change(int p,int i,int x){//change(1,i,x);
3     if(t[p].l == t[p].r) {t[p].mx = x;return;}
4     int mid = t[p].l + t[p].r >> 1;
5     if(i <= mid) change(p+p,i,x);
6     else change(p+p+1,i,x);
7     t[p].mx = max(t[p+p].mx,t[p+p+1].mx);
8 }
9
10 int query_max(int p,int l,int r){//query(1,l,r);
11     if(l <= t[p].l && r >= t[p].r) return t[p].mx;
12     int mid = t[p].l + t[p].r >> 1;
13     int ans = -1e9;
14     if(l <= mid) ans = max(ans,query(p+p,l,r));
15     if(r > mid) ans = max(ans,query(p+p+1,l,r));
16     return ans;
17 }
18
19 int query(int p,int i){//单点查询
20     if(t[p].l == t[p].r) return t[p].mx;
21     int mid = t[p].l + t[p].r >> 1;
22     if(i <= mid) return query(p<<1,i);
23     else return query(p<<1|1,i);
24 }
```

```
1 //单点修改，求区间最大子段和 https://www.luogu.com.cn/problem/SP1716
2 //单点修改，求区间最大值出现次数 https://ac.nowcoder.com/acm/contest/90296/E
3 #include <iostream>
4 #include <algorithm>
5 using namespace std;
6 const int N = 50004;
7 int n,q;
8 int a[N];
9
10 struct ST{
11     int l,r;
12     int ls,rs;//紧靠左端点和紧靠右端点的最大子段和
13     int ans,sum;//sum为区间和，ans为最长子段和
14 }t[N<<2];
15
```

```

16 void pushup(ST &p,ST &pl,ST &pr){
17     p.sum = pl.sum + pr.sum;
18     p.ls = max(pl.ls,pl.sum+pr.ls);//紧靠左端点不跨越中间和跨越中间取最大值
19     p.rs = max(pr.rs,pr.sum+pl.rs);
20     p.ans = max({pl.ans,pr.ans,pl.rs+pr.ls});//答案在{左区间答案,右区间答案,跨中间的
    最长子段}取最大值
21 }
22
23 void build(int p,int l,int r){
24     t[p] = {l,r};
25     if(l == r){
26         t[p].ls = t[p].rs = t[p].ans = t[p].sum = a[l];
27         return;
28     }
29     int mid = l + r >> 1;
30     build(p<<1,l,mid);build(p<<1|1,mid+1,r);
31     pushup(t[p],t[p<<1],t[p<<1|1]);
32 }
33
34 void change(int p,int i,int x){
35     if(t[p].l == t[p].r){
36         t[p].ls = t[p].rs = t[p].ans = t[p].sum = x;
37         return;
38     }
39     int mid = t[p].l + t[p].r >> 1;
40     if(i <= mid) change(p<<1,i,x);
41     else change(p<<1|1,i,x);
42     pushup(t[p],t[p<<1],t[p<<1|1]);
43 }
44
45 ST query(int p,int l,int r){//更加通用的query写法,返回的是一个节点包含了区间[l,r]的
    ans、sum等信息
46     if(l <= t[p].l && r >= t[p].r){
47         return t[p];
48     }
49     int mid = t[p].l + t[p].r >> 1;
50     if(r <= mid) return query(p<<1,l,r);//要查询的区间在mid左边,答案在左区间,注意l,r位
    置
51     if(l > mid) return query(p<<1|1,l,r);//要查询的区间在mid右边,答案在右区间
52     ST pl = query(p<<1,l,r);
53     ST pr = query(p<<1|1,l,r);
54     ST ans;
55     pushup(ans,pl,pr);
56     return ans;
57 }
58
59 int main(){
60     cin >> n;
61     for(int i = 1;i <= n;i++){
62         cin >> a[i];
63     }
64     build(1,1,n);

```

```

65     cin >> q;
66     while(q--){
67         int op,l,r;cin >> op >> l >> r;
68         if(op == 0){
69             change(1,l,r);
70         }
71         else{
72             cout << query(1,l,r).ans << '\n';
73         }
74     }
75 }

```

F - Palindrome Query (atcoder.jp)

单点修改，查询区间是否为回文串(字符串哈希)

$$h(abcd) = s[0] * p^3 + s[1] * p^2 + s[2] * p^1 + s[3]$$

我们只需要单点修改，求区间和即可得到子串哈希值。比较正反串的哈希值是否一致判断回文串。

```

1  #include <iostream>
2  using namespace std;
3  using ull = unsigned long long;
4  const ull P = 131;
5  const int N = 1000006;
6  int n,q;
7  ull f[N];
8  string s;
9
10 struct ST{
11     int l,r;
12     ull kl,kr;
13 }t[N<<2];
14
15 void pushup(ST &p,ST &p1,ST &pr){
16     p.kl = p1.kl + pr.kl;
17     p.kr = p1.kr + pr.kr;
18 }
19
20 void pushup(int p){
21     pushup(t[p],t[p<<1],t[p<<1|1]);
22 }
23
24 void update(ST &p,char x){
25     p.kl = f[n-p.l]*x;
26     p.kr = f[p.l-1]*x;
27 }
28
29 void build(int p,int l,int r){
30     t[p] = {l,r};
31     if(l == r){

```

```

32     update(t[p],s[l-1]);
33     return;
34 }
35 int mid = l + r >> 1;
36 build(p<<1,l,mid);build(p<<1|1,mid+1,r);
37 pushup(p);
38 }
39
40 void modify(int p,int i,char x){
41     if(t[p].l == t[p].r){
42         update(t[p],x);
43         return;
44     }
45     int mid = t[p].l + t[p].r >> 1;
46     if(i <= mid) modify(p<<1,i,x);
47     else modify(p<<1|1,i,x);
48     pushup(p);
49 }
50
51 ST query(int p,int l,int r){
52     if(l <= t[p].l && r >= t[p].r){
53         return t[p];
54     }
55     int mid = t[p].l + t[p].r >> 1;
56     if(r <= mid) return query(p<<1,l,r);
57     if(l > mid) return query(p<<1|1,l,r);
58     ST pl = query(p<<1,l,r);
59     ST pr = query(p<<1|1,l,r);
60     ST ans;
61     pushup(ans,pl,pr);
62     return ans;
63 }
64
65 int main(){
66     f[0] = 1;
67     for(int i = 1;i < N;i++){
68         f[i] = f[i-1]*P;
69     }
70
71     cin >> n >> q;
72     cin >> s;
73     build(1,1,n);
74
75     while(q--){
76         int op;cin >> op;
77         if(op == 1){
78             int x; char op; cin >> x >> op;
79             modify(1,x,op);
80         }
81         else{
82             int l,r;cin >> l >> r;
83             auto ans = query(1,l,r);

```

```

84         ull k1 = ans.k1,kr = ans.kr;
85
86         int k1 = l-1,k2 = n-r;
87         if(k1 > k2) k1 *= f[k1-k2];
88         else kr *= f[k2-k1];
89
90         if(k1 == kr){ cout << "Yes\n"; }
91         else{ cout << "No\n"; }
92     }
93 }
94 }

```

```

1 //区间修改,区间公约数 https://www.luogu.com.cn/problem/P10463
2 //利用gcd辗转相减法性质, 将a[]数组变为差分数组d[], 区间修改(a[l~r]+x)变为单点修改
  (a[l]+x,d[r-1]-x)
3 //答案为 gcd(a[l],d[l+1],...,d[r]) 的绝对值
4 #include <iostream>
5 using namespace std;
6 using ll = long long;
7 const int N = 500005;
8 int n,q;
9 ll a[N];
10
11 ll gcd(ll a,ll b){return b?gcd(b,a%b):a;}
12
13 struct ST{
14     int l,r;
15     ll dat,sum;
16 }t[N<<2];
17
18 void pushup(ST &p,ST &p1,ST &pr){
19     p.dat = gcd(p1.dat,pr.dat);
20     p.sum = p1.sum + pr.sum;
21 }
22
23 void build(int p,int l,int r){
24     t[p] = {l,r};
25     if(l == r){
26         t[p].dat = t[p].sum = a[l] - a[l-1];
27         return;
28     }
29     int mid = l + r >> 1;
30     build(p<<1,l,mid);build(p<<1|1,mid+1,r);
31     pushup(t[p],t[p<<1],t[p<<1|1]);
32 }
33
34 void add(int p,int i,ll x){
35     if(t[p].l == t[p].r){
36         t[p].dat += x;

```

```

37     t[p].sum += x;
38     return;
39 }
40 int mid = t[p].l + t[p].r >> 1;
41 if(i <= mid) add(p<<1,i,x);
42 else add(p<<1|1,i,x);
43 pushup(t[p],t[p<<1],t[p<<1|1]);
44 }
45
46 ll query_sum(int p,int l,int r){
47     if(l <= t[p].l && r >= t[p].r){
48         return t[p].sum;
49     }
50     ll ans = 0;
51     int mid = t[p].l + t[p].r >> 1;
52     if(l <= mid) ans += query_sum(p<<1,l,r);
53     if(r > mid) ans += query_sum(p<<1|1,l,r);
54     return ans;
55 }
56
57 ll query_gcd(int p,int l,int r){
58     if(l <= t[p].l && r >= t[p].r){
59         return t[p].dat;
60     }
61     ll ans = 0;
62     int mid = t[p].l + t[p].r >> 1;
63     if(l <= mid) ans = gcd(ans,query_gcd(p<<1,l,r));
64     if(r > mid) ans = gcd(ans,query_gcd(p<<1|1,l,r));
65     return ans;
66 }
67
68 int main(){
69     cin >> n >> q;
70     for(int i = 1;i <= n;i++){
71         cin >> a[i];
72     }
73     build(1,1,n);
74     while(q--){
75         char op;cin >> op;
76         if(op == 'C'){
77             ll l,r,x;cin >> l >> r >> x;
78             add(1,l,x);
79             if(r+1 <= n) add(1,r+1,-x);
80         }
81         else{
82             int l,r;cin >> l >> r;
83             cout << abs(gcd(query_gcd(1,l+1,r),query_sum(1,1,l))) << '\n';
84         }
85     }
86 }

```

区间修改，区间查询，lazy标记

通过延迟对节点信息的更改，从而减少可能不必要的操作次数。每次执行修改时，我们通过打标记的方法表明该节点对应的区间在某一次操作中被更改，但不更新该节点的子节点的信息。实质性的修改则在下一次访问带有标记的节点时才进行。叶子结点无需下放标记。

[Transformation - HDU 4578 - Virtual Judge \(vjudge.net\)](https://vjudge.net/contest/4578)

对数组 $a[]$ 实现以下操作

1 x y z 将区间 $[x,y]$ 之间的每个元素增加 z 。 $1 \leq z \leq 10000$

2 x y z 将区间 $[x,y]$ 之间的每个元素乘以 z 。 $1 \leq z \leq 10000$

3 x y z 将区间 $[x,y]$ 之间的每个元素变为 z 。 $1 \leq z \leq 10000$

4 x y z 输出 $a_x^z + a_{x+1}^z + \dots + a_y^z$ 对 10007 取模后的结果。 $1 \leq z \leq 3$

```
1  #include <iostream>
2
3  const int mod = 10007;
4  const int N = 100005;
5  int n,m;
6
7  struct ST{
8      int l,r;
9      long long d[4];
10     long long add,mul;
11 }t[N<<2];
12
13 void build(int p,int l,int r){
14     t[p] = {l,r};
15     if(l == r){
16         t[p].add = 0;
17         t[p].mul = 1;
18         return;
19     }
20     int mid = l + r >> 1;
21     build(p<<1,l,mid);build(p<<1|1,mid+1,r);
22 }
23
24 void pushup(ST &p,ST &p1,ST &pr){
25     p.d[1] = (p1.d[1] + pr.d[1]) % mod;
26     p.d[2] = (p1.d[2] + pr.d[2]) % mod;
27     p.d[3] = (p1.d[3] + pr.d[3]) % mod;
28 }
29
30 void pushup(int p){
31     pushup(t[p],t[p<<1],t[p<<1|1]);
32 }
33
34 void update(ST &p,long long add,long long mul){//注意答案、数据的更新顺序
35     int len = p.r - p.l + 1;
36     p.add = (p.add * mul + add) % mod;
37     p.mul = (p.mul * mul) % mod;
```

```

38     p.d[3] = (mul*mul%mod*mul%mod*p.d[3]%mod + 3*mul*mul%mod*add%mod*p.d[2]%mod
+ 3*add*add%mod*mul%mod*p.d[1]%mod + add*add%mod*add%mod*len%mod) % mod;
39     p.d[2] = (p.d[2]*mul%mod*mul%mod + 2*mul*add%mod*p.d[1]%mod +
add*add%mod*len%mod) % mod;
40     p.d[1] = (p.d[1]*mul + len*add) % mod;
41 }
42
43 void pushdown(int p){
44     auto &add = t[p].add, &mul = t[p].mul;
45     if(add || mul != 1){//同时下传所有标记
46         update(t[p<<1],add,mul);
47         update(t[p<<1|1],add,mul);
48         add = 0,mul = 1;
49     }
50 }
51
52 void modify(int p,int l,int r,int x,int op){//用当前标记更新答案(和更新其它标记(如果有
且需要更新))
53     if(l <= t[p].l && r >= t[p].r){
54         if(op == 1){//add
55             update(t[p],x,1);
56         }
57         if(op == 2){//mul
58             update(t[p],0,x);
59         }
60         if(op == 3){//same 相当于区间乘0再加x
61             update(t[p],x,0);
62         }
63         return;
64     }
65     pushdown(p);
66     int mid = t[p].l + t[p].r >> 1;
67     if(l <= mid) modify(p<<1,l,r,x,op);
68     if(r > mid) modify(p<<1|1,l,r,x,op);
69     pushup(p);
70 }
71
72 ST query(int p,int l,int r){//比较通用的query写法
73     if(l <= t[p].l && r >= t[p].r){
74         return t[p];
75     }
76     pushdown(p);
77     int mid = t[p].l + t[p].r >> 1;
78     if(r <= mid) return query(p<<1,l,r);
79     if(l > mid) return query(p<<1|1,l,r);
80     ST pl = query(p<<1,l,r),pr = query(p<<1|1,l,r),ans;
81     pushup(ans,pl,pr);
82     return ans;
83 }
84
85 void sol(){
86     build(1,1,n);

```

```

87     while(m--){
88         int op,x,y,z;std::cin >> op >> x >> y >> z;
89         if(op == 4)std::cout << query(1,x,y).d[z] << '\n';
90         else modify(1,x,y,z,op);
91     }
92 }
93
94 int main(){
95     std::ios::sync_with_stdio(false);std::cin.tie(0);
96     while(std::cin >> n >> m,n|m) sol();
97 }

```

[SP2713 GSS4 - Can you answer these queries IV - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/SP2713)

区间开平方，求区间和

对于区间开平方，由于我们无法在不更新叶子节点时更新区间，因此无法使用lazy标记优化区间修改。考虑到每个数最多被开平方有限次之后就会变为1，于是我们在修改时，一直修改到叶子节点 或者 当前区间所有数都为1。

```

1  const int N = 100005;
2  int n,q;
3  long long a[N];
4
5  struct ST{
6      int l,r;
7      long long dat; // 若值域含0,则可以开一个cnt统计区间 <= 1的个数
8  }t[N<<2];
9
10 void pushup(ST &p,ST &pl,ST &pr){
11     p.dat = pl.dat + pr.dat;
12 }
13
14 void pushup(int p){
15     pushup(t[p],t[p<<1],t[p<<1|1]);
16 }
17
18 void build(int p,int l,int r){
19     t[p] = {l,r};
20     if(l == r){
21         t[p].dat = a[l];
22         return;
23     }
24     int mid = l + r >> 1;
25     build(p<<1,l,mid);build(p<<1|1,mid+1,r);
26     pushup(p);
27 }
28
29 void modify(int p,int l,int r){
30     if(t[p].dat == t[p].r - t[p].l + 1) { // 当前区间所有数都为1时,直接返回

```

```

31         return;
32     }
33     if(t[p].l == t[p].r) { //当前为叶子节点时,直接修改
34         t[p].dat = std::sqrt(t[p].dat);
35         return;
36     }
37     int mid = t[p].l + t[p].r >> 1;
38     if(l <= mid) modify(p<<1,l,r);
39     if(r > mid) modify(p<<1|1,l,r);
40     pushup(p);
41 }
42
43 ST query(int p,int l,int r){
44     if(l <= t[p].l && r >= t[p].r){
45         return t[p];
46     }
47     int mid = t[p].l + t[p].r >> 1;
48     if(r <= mid) return query(p<<1,l,r);
49     if(l > mid) return query(p<<1|1,l,r);
50     ST pl = query(p<<1,l,r),pr = query(p<<1|1,l,r),ans;
51     pushup(ans,pl,pr);
52     return ans;
53 }
54
55 void sol(){
56     for(int i = 1;i <= n;i++) std::cin >> a[i];
57     build(1,1,n);
58     std::cin >> q;
59     while(q--){
60         int op,l,r;std::cin >> op >> l >> r;
61         if(l > r) std::swap(l,r);
62         if(op == 0) modify(1,l,r);
63         else std::cout << query(1,l,r).dat << '\n';
64     }
65 }

```

权值线段树

对权值作为维护对象而开的线段树，每个点存的是区间内对应数字的某种值(如出现次数等)，操作跟线段树类似。

诺题目允许离线，则离散化后空间复杂度为 $O(N)$ ，否则需要开到 $O(\max_val)$ 的大小。

[P3369 【模板】普通平衡树 - 洛谷\(luogu.com.cn\)](https://www.luogu.com.cn/problem/P3369)

与权值树状数组实现类似。

```

1 #include <iostream>
2 #include <algorithm>

```

```

3  #include <vector>
4  using namespace std;
5  const int N = 100005;
6  int op[N],a[N];
7  vector<int>hs;
8
9  struct st{
10     int l,r;
11     int dat;
12 }t[N<<2];
13
14 void pushup(int p){
15     t[p].dat = t[p<<1].dat + t[p<<1|1].dat;
16 }
17
18 void build(int p,int l,int r){
19     t[p] = {l,r};
20     if(l == r){
21         t[p].dat = 0;
22         return;
23     }
24     int mid = l + r >> 1;
25     build(p<<1,l,mid);build(p<<1|1,mid+1,r);
26     pushup(p);
27 }
28
29 void modify(int p,int l,int r,int x){//单点修改
30     if(l <= t[p].l && r >= t[p].r){
31         if(t[p].dat + x >= 0) t[p].dat += x;
32         return;
33     }
34     int mid = t[p].l + t[p].r >> 1;
35     if(l <= mid) modify(p<<1,l,r,x);
36     if(r > mid) modify(p<<1|1,l,r,x);
37     pushup(p);
38 }
39
40
41 int rnk(int p,int l,int r){//查询区间[l,r]的和
42     if(l <= t[p].l && r >= t[p].r){
43         return t[p].dat;
44     }
45     int mid = t[p].l + t[p].r >> 1;
46     int ans = 0;
47     if(l <= mid) ans += rnk(p<<1,l,r);
48     if(r > mid) ans += rnk(p<<1|1,l,r);
49     return ans;
50 }
51
52 int kth(int p,int k){//线段树上二分,查询全局第k小的数的下标(只能查询全局)
53     if(t[p].l == t[p].r){
54         return t[p].l;

```

```

55     }
56     if(k <= t[p<<1].dat) return kth(p<<1,k);
57     else return kth(p<<1|1,k-t[p<<1].dat);
58 }
59
60 int main(){
61     int q;cin >> q;
62     for(int i = 1;i <= q;i++){
63         cin >> op[i] >> a[i];
64         if(op[i] == 4) continue;
65         hs.emplace_back(a[i]);
66     }
67
68     hs.emplace_back(-1e9);
69     sort(hs.begin(),hs.end());
70     hs.erase(unique(hs.begin(),hs.end()),hs.end());
71     int m = hs.size()-1;
72     for(int i = 1;i <= q;i++){
73         if(op[i] == 4) continue;
74         a[i] = lower_bound(hs.begin(),hs.end(),a[i])-hs.begin();
75     }
76
77     build(1,1,m);
78     for(int i = 1;i <= q;i++){
79         if(op[i] == 1){//插入一个数x
80             modify(1,a[i],a[i],1);
81         }
82         if(op[i] == 2){//删除一个数x(如有多个,只删除一个)
83             modify(1,a[i],a[i],-1);
84         }
85         if(op[i] == 3){//a[i]的排名,即区间[1,a[i]-1]的数的个数再加1
86             if(a[i] == 1) {cout << 1 << '\n';continue;}
87             cout << rnk(1,1,a[i]-1)+1 << '\n';
88         }
89         if(op[i] == 4){//查询全局第k小的数(下标从1开始)
90             cout << hs[kth(1,a[i])] << '\n';
91         }
92         if(op[i] == 5){//查找x的前驱(若不存在则返回x)
93             int rk = rnk(1,1,a[i]-1);
94             cout << hs[kth(1,rk)] << '\n';
95         }
96         if(op[i] == 6){//查找x的后继(若不存在则返回x)
97             int rk = rnk(1,1,a[i])+1;
98             cout << hs[kth(1,rk)] << '\n';
99         }
100     }
101 }

```

线段树上二分

[U420425 线段树二分 2 - 洛谷](#)

查询区间 $[l, r]$ 第一个前缀和 $\geq x$ 的下标，若都符合则输出 $r+1$ (类似于 `lower_bound`)

```
1  #include <iostream>
2
3  const int N = 1000006;
4  int n, q;
5  long long a[N];
6
7  struct ST {
8      int l, r;
9      long long sum, add;
10 } t[N << 2];
11
12 void pushup(ST &p, ST &pl, ST &pr) {
13     p.sum = pl.sum + pr.sum;
14 }
15
16 void pushup(int p) {
17     pushup(t[p], t[p << 1], t[p << 1 | 1]);
18 }
19
20 void update(ST &p, long long x, int op) {
21     if (op == 1) {
22         p.sum += (p.r - p.l + 1) * x;
23         p.add += x;
24     }
25     if (op == 4) {
26         p.sum = x;
27         p.add = 0;
28     }
29 }
30
31 void pushdown(int p) {
32     long long &x = t[p].add;
33     if (x) {
34         update(t[p << 1], x, 1);
35         update(t[p << 1 | 1], x, 1);
36         x = 0;
37     }
38 }
39
40 void build(int p, int l, int r) {
41     t[p] = {l, r};
42     if (l == r) {
43         t[p].sum = a[l];
44         return;
45     }
46     int mid = l + r >> 1;
```

```

47     build(p << 1, l, mid); build(p << 1 | 1, mid + 1, r);
48     pushup(p);
49 }
50
51 void modify(int p, int l, int r, long long x, int op) {
52     if (l <= t[p].l && r >= t[p].r) {
53         update(t[p], x, op);
54         return;
55     }
56     pushdown(p);
57     int mid = t[p].l + t[p].r >> 1;
58     if (l <= mid) modify(p << 1, l, r, x, op);
59     if (r > mid) modify(p << 1 | 1, l, r, x, op);
60     pushup(p);
61 }
62
63 ST query(int p, int l, int r) {
64     if (l <= t[p].l && r >= t[p].r) {
65         return t[p];
66     }
67     pushdown(p);
68     int mid = t[p].l + t[p].r >> 1;
69     if (r <= mid) return query(p << 1, l, r);
70     if (l > mid) return query(p << 1 | 1, l, r);
71     ST pl = query(p << 1, l, r), pr = query(p << 1 | 1, l, r), ans;
72     pushup(ans, pl, pr);
73     return ans;
74 }
75
76 int findFirst(int p, int l, int r, ST &pre, long long x) { //lower_bound
77     if (l > t[p].r || r < t[p].l) return -1;
78     if (l <= t[p].l && r >= t[p].r) {
79         if (pre.sum + t[p].sum < x) {
80             pre.sum += t[p].sum;
81             return -1;
82         }
83     }
84     if (t[p].l == t[p].r) return t[p].l;
85     pushdown(p);
86
87     int res = findFirst(p << 1, l, r, pre, x);
88     if (res != -1) return res;
89     return findFirst(p << 1 | 1, l, r, pre, x);
90 }
91
92 int findFirst(int l, int r, long long x) {
93     ST pre{};
94     return findFirst(1, l, r, pre, x);
95 }
96
97 int main() {
98     std::ios::sync_with_stdio(false); std::cin.tie(0);

```

```

99     std::cin >> n >> q;
100     for (int i = 1; i <= n; i++) {
101         std::cin >> a[i];
102     }
103     build(1, 1, n);
104
105     while (q--) {
106         int op; std::cin >> op;
107         if (op == 1) { // 区间加
108             int l, r, k; std::cin >> l >> r >> k;
109             modify(1, l, r, k, op);
110         }
111         if (op == 2) { // 求区间和
112             int l, r; std::cin >> l >> r;
113             std::cout << query(1, l, r).sum << '\n';
114         }
115         if (op == 3) { // 求区间第一个前缀和 >= k 的下标, 若都大于k则输出r+1
116             int l, r, k; std::cin >> l >> r >> k;
117             int res = findFirst(1, r, k);
118             std::cout << (res != -1 ? res : r + 1) << '\n';
119         }
120         if (op == 4) { // 单点修改
121             int l, d; std::cin >> l >> d;
122             modify(1, l, l, d, op);
123         }
124     }
125 }

```

可持久化线段树

[【模板】可持久化线段树 1 \(可持久化数组\) - 洛谷](#)

维护一个长度为 N 的数组，支持以下两种操作。

- 在某个历史版本上修改某一个位置的值
- 查询某个历史版本上某一个位置的值

```

1  #include <iostream>
2
3  const int N = 1000006;
4  int n, q;
5  int a[N];
6
7  struct ST {
8      int l, r;
9      int val;

```

```

10 }t[N << 5];
11
12 int root[N << 5], tot;
13
14 int build(int l, int r) {
15     int p = ++tot;
16     if (l == r) {
17         t[p].val = a[l];
18         return p;
19     }
20     int mid = l + r >> 1;
21     t[p].l = build(l, mid);
22     t[p].r = build(mid + 1, r);
23     return p;
24 }
25
26 int update(int rt, int l, int r, int pos, int val) {
27     int p = ++tot;
28     t[p] = t[rt];
29     if (l == r) {
30         t[p].val = val;
31         return p;
32     }
33     int mid = l + r >> 1;
34     if (pos <= mid) t[p].l = update(t[p].l, l, mid, pos, val);
35     else t[p].r = update(t[p].r, mid + 1, r, pos, val);
36     return p;
37 }
38
39 int query(int p, int l, int r, int pos) {
40     if (l == r) {
41         return t[p].val;
42     }
43     int mid = l + r >> 1;
44     if (pos <= mid) return query(t[p].l, l, mid, pos);
45     else return query(t[p].r, mid + 1, r, pos);
46 }
47
48 int main() {
49     std::ios::sync_with_stdio(false); std::cin.tie(0);
50     std::cin >> n >> q;
51     for (int i = 1; i <= n; i++) {
52         std::cin >> a[i];
53     }
54
55     root[0] = build(1, n);
56
57     for (int i = 1; i <= q; i++) {
58         int version, op, pos;
59         std::cin >> version >> op >> pos;
60         if (op == 1) {
61             int val; std::cin >> val;

```

```

62         root[i] = update(root[version], 1, n, pos, val);
63     }
64     if (op == 2) {
65         root[i] = root[version];
66         std::cout << query(root[i], 1, n, pos) << '\n';
67     }
68 }
69 }

```

静态区间第K小【模板】可持久化线段树 2 - 洛谷

给定长度为 N 的数组， Q 次询问：求区间 $[l, r]$ 的第 k 小的值。

不支持修改操作

子树 $root[i]$ 存的是区间 $[1, i]$ 中，所有数出现的信息，即一个值域集合。

利用 $root[r]$ 的信息减去 $root[l-1]$ 的信息，即可得到区间 $[l, r]$ 的信息。

```

1  #include <bits/stdc++.h>
2
3  const int N = 200005;
4  int n, q;
5  int a[N];
6  std::vector<int> hs(1, -2e9); //离散化
7
8  struct ST {
9      int l, r; //l,r记录的是左,右子节点编号
10     int val;
11 } t[N << 5]; //空间至少开到*20
12
13 int root[N << 5], tot;
14
15 int build(int l, int r) {
16     int p = ++tot;
17     t[p].val = 0;
18     if (l == r) { return p; }
19     int mid = l + r >> 1;
20     t[p].l = build(l, mid);
21     t[p].r = build(mid + 1, r);
22     return p;
23 }
24
25 int insert(int rt, int l, int r, int pos) {
26     int p = ++tot;
27     t[p] = t[rt];
28     t[p].val = t[rt].val + 1;
29     if (l == r) { return p; }
30     int mid = l + r >> 1;

```

```

31     if (pos <= mid) t[p].l = insert(t[p].l, 1, mid, pos);
32     else t[p].r = insert(t[p].r, mid + 1, r, pos);
33     return p;
34 }
35
36 int kth(int p1, int p2, int l, int r, int k) {
37     if (l == r) return l;
38     int sum = t[t[p2].l].val - t[t[p1].l].val; //通过区间减法得到左儿子中所存储的数值个数
39     int mid = l + r >> 1;
40     if (k <= sum) return kth(t[p1].l, t[p2].l, l, mid, k); //若k小于等于sum,则说明第k小的数在左儿子中
41     else return kth(t[p1].r, t[p2].r, mid + 1, r, k - sum); //否则说明在右儿子中
42 }
43
44 int main() {
45     std::cin >> n >> q;
46     for (int i = 1; i <= n; i++) {
47         std::cin >> a[i];
48         hs.push_back(a[i]);
49     }
50
51     std::sort(hs.begin() + 1, hs.end());
52     hs.erase(std::unique(hs.begin() + 1, hs.end()), hs.end());
53     for (int i = 1; i <= n; i++) {
54         a[i] = std::lower_bound(hs.begin() + 1, hs.end(), a[i]) - hs.begin();
55     }
56
57     for (int i = 1; i <= n; i++) {
58         root[i] = insert(root[i - 1], 1, hs.size(), a[i]);
59     }
60
61     while (q--) {
62         int l, r, k; std::cin >> l >> r >> k;
63         int ans = kth(root[l - 1], root[r], 1, hs.size(), k);
64         std::cout << hs[ans] << '\n';
65     }
66 }

```

[树上计数 - AcWing题库](#)

给定一颗 N 个节点的树，每个节点有一个权值，回答 Q 次询问：
求 (u, v) 简单路径上第 k 小的点权。

类似于树上差分，我们DFS时建树令 `root[x] = insert(root[father], 1, n, a[x])`,

对于每个询问 (x, y, k) 令 $z = \text{lca}(x, y)$ ，则当前询问答案 `ans =`

`query(root[x], root[y], root[z], root[fa[z]], 1, n, k)`,

左儿子 `sum = t[t[p1].l].val + t[t[p2].l].val - t[t[p3].l].val - t[t[p4].l].val`。

```
1 #include <iostream>
```

```

2  #include <vector>
3  #include <algorithm>
4  #include <cstring>
5
6  const int N = 100005;
7  int n, q;
8  int a[N];
9  std::vector<int> hs(1, -2e9);
10
11 const int M = N << 1;
12 int h[N], e[M], ne[M], idx;
13 void add(int u, int v) {
14     e[idx] = v, ne[idx] = h[u], h[u] = idx++;
15 }
16
17 struct ST {
18     int l, r;
19     int val;
20 }t[N << 5];
21 int root[N], tot;
22
23 int insert(int rt, int l, int r, int pos) {
24     int p = ++tot;
25     t[p] = t[rt];
26     t[p].val = t[rt].val + 1;
27     if (l == r) return p;
28     int mid = l + r >> 1;
29     if (pos <= mid) t[p].l = insert(t[p].l, l, mid, pos);
30     else t[p].r = insert(t[p].r, mid + 1, r, pos);
31     return p;
32 }
33
34 int query(int p1, int p2, int p3, int p4, int l, int r, int k) {
35     if (l == r) return l;
36     int sum = t[t[p1].l].val + t[t[p2].l].val - t[t[p3].l].val -
37     t[t[p4].l].val;
38     int mid = l + r >> 1;
39     if (k <= sum) return query(t[p1].l, t[p2].l, t[p3].l, t[p4].l, l, mid, k);
40     else return query(t[p1].r, t[p2].r, t[p3].r, t[p4].r, mid + 1, r, k -
41     sum);
42 }
43
44 int siz[N], son[N], dep[N], fa[N];
45 void dfs1(int x, int father) {
46     siz[x] = 1;
47     fa[x] = father;
48     dep[x] = dep[father] + 1;
49     root[x] = insert(root[father], 1, hs.size(), a[x]);
50     for (int i = h[x]; ~i; i = ne[i]) {
51         int y = e[i];
52         if (y != father) {
53             dfs1(y, x);
54         }
55     }
56 }

```

```

52         siz[x] += siz[y];
53         if (siz[y] > siz[son[x]]) son[x] = y;
54     }
55 }
56 }
57
58 int dfn[N], rnk[N], top[N], idn;
59 void dfs2(int x, int father) {
60     top[x] = father;
61     dfn[x] = ++idn;
62     rnk[idn] = x;
63
64     if (!son[x]) return;
65     dfs2(son[x], father);
66     for (int i = h[x]; ~i; i = ne[i]) {
67         int y = e[i];
68         if (y != son[x] && y != fa[x]) {
69             dfs2(y, y);
70         }
71     }
72 }
73
74 int lca(int x, int y) {
75     while (top[x] != top[y]) {
76         if (dep[top[x]] > dep[top[y]]) x = fa[top[x]];
77         else y = fa[top[y]];
78     }
79     return dfn[x] < dfn[y] ? x : y;
80 }
81
82 int main() {
83     std::ios::sync_with_stdio(false); std::cin.tie(0);
84     std::memset(h, -1, sizeof h);
85     std::cin >> n >> q;
86     for (int i = 1; i <= n; i++) {
87         std::cin >> a[i];
88         hs.push_back(a[i]);
89     }
90
91     std::sort(hs.begin() + 1, hs.end());
92     hs.erase(std::unique(hs.begin() + 1, hs.end()), hs.end());
93     for (int i = 1; i <= n; i++) {
94         a[i] = std::lower_bound(hs.begin() + 1, hs.end(), a[i]) - hs.begin();
95     }
96
97     for (int i = 1; i < n; i++) {
98         int u, v; std::cin >> u >> v;
99         add(u, v);
100        add(v, u);
101    }
102
103    dfs1(1, 0);

```

```

104     dfs2(1, 1);
105
106     int last = 0;
107     while (q--) {
108         int x, y, k; std::cin >> x >> y >> k;
109         x ^= last; //本题强制在线
110         int z = lca(x, y);
111         int ans = query(root[x], root[y], root[z], root[fa[z]], 1, hs.size(),
112 k);
113         std::cout << hs[ans] << '\n';
114         last = hs[ans];
115     }
116 }

```

标记永久化

减少空间占用，标记永久化适合满足可交换、可结合的操作，如：区间加、区间乘。不适合：区间赋值、区间除法、区间最值、复杂复合操作等。

应用范围比较窄，在其适用的场景中无可替代，但在不合适的场景中会变得很难使用。

[To the moon - HDU 4348 - Virtual Judge](#)

维护一个长度为 N 的序列，版本号初始为0，支持以下操作：

- **C l r d** 区间 $[l, r]$ 的每个数加 d ，版本号 $+1$
- **Q l r** 查询当前版本区间 $[l, r]$ 的数之和
- **H l r t** 查询 t 版本区间 $[l, r]$ 的数之和
- **B t** 回滚到 t 版本

```

1  #include <iostream>
2
3  const int N = 100005;
4  int n, q;
5  long long a[N];
6
7  struct ST {
8      int l, r;
9      long long sum; //sum存储的是不考虑当前add标记时的区间和
10     long long add; //永久化标记,不会下传到子节点
11 } t[N << 5];
12
13 int root[N << 5], tot;
14
15 int build(int l, int r) {
16     int p = ++tot;
17     t[p].add = t[p].sum = 0;

```

```

18     if (l == r) {
19         t[p].sum = a[l];
20         return p;
21     }
22     int mid = l + r >> 1;
23     t[p].l = build(l, mid);
24     t[p].r = build(mid + 1, r);
25     t[p].sum = t[t[p].l].sum + t[t[p].r].sum;
26     return p;
27 }
28
29 int modify(int rt, int l, int r, int ql, int qr, int x) {
30     int p = ++tot;
31     t[p] = t[rt];
32     if (ql <= l && qr >= r) { //区间完全被覆盖,只更新add标记,不更新sum
33         t[p].add += x;
34         return p;
35     }
36     //部分覆盖时,递归修改子节点,然后更新当前节点的sum值
37     int mid = l + r >> 1;
38     if (ql <= mid) t[p].l = modify(t[p].l, l, mid, ql, qr, x);
39     if (qr > mid) t[p].r = modify(t[p].r, mid + 1, r, ql, qr, x);
40     t[p].sum += (std::min(r, qr) - std::max(l, ql) + 1) * x;
41     return p;
42 }
43
44 long long query(int p, int l, int r, int ql, int qr) {
45     if (ql <= l && qr >= r) { //完全覆盖,返回sum + add标记对区间的影响
46         return t[p].sum + t[p].add * (r - l + 1);
47     }
48     //递归子区间
49     int mid = l + r >> 1;
50     long long ans = 0;
51     if (ql <= mid) ans += query(t[p].l, l, mid, ql, qr);
52     if (qr > mid) ans += query(t[p].r, mid + 1, r, ql, qr);
53     //加上当前add标记对查询区间影响
54     return ans + t[p].add * (std::min(r, qr) - std::max(l, ql) + 1);
55 }
56
57 void sol() {
58     tot = 0;
59     for (int i = 1; i <= n; i++) {
60         std::cin >> a[i];
61     }
62
63     root[0] = build(1, n);
64
65     int tim = 0;
66     while (q--) {
67         char op; std::cin >> op;
68         if (op == 'C') {
69             int l, r, d; std::cin >> l >> r >> d;

```

```

70         tim++;
71         root[tim] = modify(root[tim - 1], 1, n, 1, r, d);
72     }
73     if (op == 'Q') {
74         int l, r; std::cin >> l >> r;
75         std::cout << query(root[tim], 1, n, l, r) << '\n';
76     }
77     if (op == 'H') {
78         int l, r, ver; std::cin >> l >> r >> ver;
79         std::cout << query(root[ver], 1, n, l, r) << '\n';
80     }
81     if (op == 'B') {
82         int ver; std::cin >> ver;
83         tim = ver;
84     }
85 }
86 }
87
88 int main() {
89     std::ios::sync_with_stdio(false); std::cin.tie(0);
90     while (std::cin >> n >> q) sol();
91 }

```

霍夫曼树

从根结点到各叶结点的路径长度与相应叶节点权值的乘积之和称为 **树的带权路径长度 (Weighted Path Length of Tree, WPL)** 。

设 w_i 为二叉树第 i 个叶结点的权值, l_i 为从根结点到第 i 个叶结点的路径长度, 则 WPL 计算公式如下:

$$WPL = \sum_{i=1}^n w_i l_i$$

(也等于所有非叶子节点之和) 对于给定一组具有确定权值的叶结点, 可以构造出不同的树, 其中, **WPL 最小的树** 称为 **霍夫曼树 (Huffman Tree)** 。霍夫曼树**不唯一**。

如果每个字符的 **使用频率相等**, 那么等长编码无疑是空间效率最高的编码方法, 而如果字符出现的频率不同, 则可以让频率高的字符采用尽可能短的编码, 频率低的字符采用尽可能长的编码, 来构造出一种 **不等长编码**, 从而获得更好的空间效率。

在设计不等长编码时, 要考虑解码的唯一性, 如果一组编码中任一编码都不是其他任何一个编码的前缀, 那么称这组编码为 **前缀编码**, 其保证了编码被解码时的唯一性。

霍夫曼树可用于构造 **最短的前缀编码**, 即 **霍夫曼编码 (Huffman Code)**

不建树直接求n个节点k叉树的最小WPL

让n满足 $((n-1) \bmod (k-1) == 0)$, 否则不断添0, $n++$;

建立小根堆, 每次取堆顶的k个求和s, 再将s入堆, 直到小根堆只剩一个元素

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4  priority_queue<int,vector<int>,greater<int>>pq;
5
6  int main(){
7      int n,k;cin >> n >> k;
8      for(int i = 1;i <= n;i++){
9          int x;cin >> x;
10         pq.push(x);
11     }
12     while((n-1)%(k-1) != 0){
13         pq.push(0);
14         n++;
15     }
16
17     int ans = 0;
18     while(pq.size() > 1){
19         int sum = 0;
20         for(int i = 0;i < k;i++){
21             int x = pq.top();
22             pq.pop();
23             sum += x;
24         }
25         pq.push(sum);
26         ans += sum;
27     }
28     cout << ans;
29 }
```

```
1  //合并果子加强版: https://www.luogu.com.cn/problem/P6033
2  //简化版为n个节点2叉树的WPL;  $n \leq 1e5$ 
3  //加强版 $n \leq 1e7$ , 但a[i]比较小, 可以用O(N)的桶排
4  //使用两个queue代替优先队列(保证单调性)
5  #include <iostream>
6  #include <queue>
7  using namespace std;
8  using ll = long long;
9  const int N = 100005;
10 int a[N],idx;
11 queue<ll>q1,q2;//q1为排序后的原数组, q2为合并的果子数组, 可以证明q2为单调递增
12
13 int read(){
14     int x = 0,f = 1;
15     char ch = getchar();
16     while(!isdigit(ch)){ if(ch == '-') f = -1; ch = getchar(); }
```

```

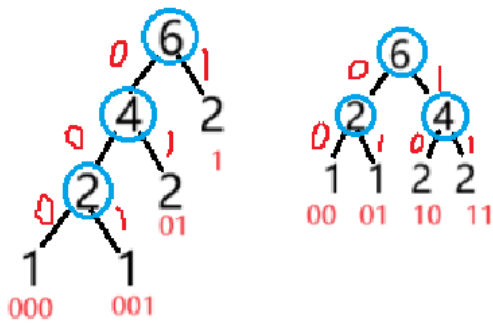
17     while(isdigit(ch)){ x = x*10 + ch - '0'; ch = getchar(); }
18     return x*f;
19 }
20
21 ll get_top(){
22     ll x;
23     if(q2.empty() || (q1.size() && q1.front() < q2.front())){
24         x = q1.front(); q1.pop();
25     }
26     else{
27         x = q2.front(); q2.pop();
28     }
29     return x;
30 }
31
32 int main(){
33     int n;cin >> n;
34     for(int i = 1;i <= n;i++){
35         int x = read();
36         a[x]++; //直接使用O(N)的桶排
37     }
38     for(int i = 1;i < N;i++){
39         for(int j = 0;j < a[i];j++){
40             q1.emplace(i);
41         }
42     }
43
44     ll ans = 0;
45     while(q1.size() + q2.size() > 1){
46         ll x = get_top(); //每次从q1和q2中选最小的两个取出
47         ll y = get_top();
48         ans += x+y;
49         q2.push(x+y); //累加到答案后，加入q2队尾
50     }
51     cout << ans;
52 }

```

[NOI2015 荷马史诗](#)

求最短WPL，并且要让最长的霍夫曼编码最短

只需在合并时，对于权值相同的节点，优先考虑当前深度最小(已合并次数最少)的进行合并即可



两种霍夫曼树WPL均为12，但右图最长编码更小

```

1  #include <iostream>
2  #include <queue>
3  using namespace std;
4  using ll = long long;
5  using pii = pair<ll,ll>;
6  int n,k;
7  priority_queue<pii,vector<pii>,greater<pii>>pq;
8  ll ans;
9
10 int main(){
11     cin >> n >> k;
12     for(int i = 1;i <= n;i++){
13         ll x;cin >> x;
14         pq.push({x,0});
15     }
16
17     while((n-1)%(k-1) != 0){
18         pq.push({0,0});
19         n++;
20     }
21
22     while(pq.size() > 1){
23         ll sum = 0;
24         ll dep = 0;
25         for(int i = 0;i < k;i++){
26             auto [x,y] = pq.top();
27             pq.pop();
28             sum += x;
29             dep = max(dep,y);
30         }
31         pq.push({sum,dep+1});
32         ans += sum;
33     }
34     cout << ans << endl << pq.top().second << endl;
35 }

```

平衡树

Splay树

Splay 树，或 **伸展树**，是一种平衡二叉查找树，它通过 **伸展 (splay) 操作** 不断将某个节点旋转到根节点，使得整棵树仍然满足二叉查找树的性质，能够在均摊 $O(\log N)$ 时间内完成插入、查找和删除操作，并且保持平衡而不至于退化为链。最为强大的功能之一是支持高效的区间操作，例如区间反转、删除、插入，求和，求最值。

Splay 树是一棵二叉查找树，查找某个值时满足性质：左子树任意节点的值 < 根节点的值 < 右子树任意节点的值。

本文使用数组模拟指针来实现 Splay 树，需要维护如下信息：

rt	id	fa[i]	ch[i][0/1]	val[i]	cnt[i]	sz[i]
根节点编号	已使用节点个数	父亲	左右儿子编号	节点权值	权值出现次数	子树大小

初始化时，所有信息都置零即可。

平衡树操作

splay_treet		
<code>t.insert(v)</code>	插入值v	
<code>t.remove(v)</code>	删除值v	只删除一个
<code>t.find_rank(v)</code>	查询值v的排名	1_idx
<code>t.find_kth(k)</code>	查找第k小的数	1_idx, 没有的返回-1
<code>t.find_prev(v)</code>	查找v的前驱	小于v的最大值，没有则返回-1
<code>t.find_next(v)</code>	查找v的后继	大于v的最小值，没有则返回-1

```
1 //普通平衡树 https://www.luogu.com.cn/problem/P3369
2 #include <iostream>
3
4 namespace Splay_Tree{
5     using T = int; //val_type
6     constexpr int N = 100005;
7     int id, rt;
8     int fa[N], cnt[N], sz[N], ch[N][2];
9     T val[N];
10
11     struct splay_tree{
```

```

12     bool dir(int x) { return x == ch[fa[x]][1]; } //判断x是父亲节点的左/右儿子
13
14     void push_up(int x) { sz[x] = cnt[x] + sz[ch[x][0]] + sz[ch[x][1]];
    } //在改变节点位置后,根据子节点信息更新节点x的信息
15
16     void rotate(int x) { //旋转操作
17         int y = fa[x], z = fa[y];
18         bool r = dir(x);
19         ch[y][r] = ch[x][!r];
20         ch[x][!r] = y;
21         if (z) ch[z][dir(y)] = x;
22         if (ch[y][r]) fa[ch[y][r]] = y;
23         fa[y] = x;
24         fa[x] = z;
25         push_up(y);
26         push_up(x);
27     }
28
29     void splay(int& z, int x) { //伸展操作
30         int w = fa[z];
31         for (int y; (y = fa[x]) != w; rotate(x)) {
32             if (fa[y] != w) rotate(dir(x) == dir(y) ? y : x);
33         }
34         z = x;
35     }
36
37     void find(int& z, T v) {
38         int x = z, y = fa[x];
39         for (; x && val[x] != v; x = ch[y = x][v > val[x]]);
40         splay(z, x ? x : y);
41     }
42
43     void loc(int& z, int k) {
44         int x = z;
45         for (;;) {
46             if (sz[ch[x][0]] >= k) {
47                 x = ch[x][0];
48             } else if (sz[ch[x][0]] + cnt[x] >= k) {
49                 break;
50             } else {
51                 k -= sz[ch[x][0]] + cnt[x];
52                 x = ch[x][1];
53             }
54         }
55         splay(z, x);
56     }
57
58     int merge(int x, int y) { //合并两颗Splay树
59         if (!x || !y) return x | y;
60         loc(y, 1);
61         ch[y][0] = x;
62         fa[x] = y;

```

```

63         push_up(y);
64         return y;
65     }
66
67     void insert(T v) { //插入值v
68         int x = rt, y = 0;
69         for (; x && val[x] != v; x = ch[y = x][v > val[x]]);
70         if (x) {
71             ++cnt[x];
72             ++sz[x];
73         } else {
74             x = ++id;
75             val[x] = v;
76             cnt[x] = sz[x] = 1;
77             fa[x] = y;
78             if (y) ch[y][v > val[y]] = x;
79         }
80         splay(rt, x);
81     }
82
83     bool remove(T v) { //删除值v
84         find(rt, v);
85         if (!rt || val[rt] != v) return false;
86         --cnt[rt];
87         --sz[rt];
88         if (!cnt[rt]) {
89             int x = ch[rt][0];
90             int y = ch[rt][1];
91             fa[x] = fa[y] = 0;
92             rt = merge(x, y);
93         }
94         return true;
95     }
96
97     int find_rank(T v) { //查询值v的排名
98         find(rt, v);
99         return sz[ch[rt][0]] + (val[rt] < v ? cnt[rt] : 0) + 1;
100     }
101
102     T find_kth(int k) { //查找第k小的值(1_idx)
103         if (k > sz[rt]) return -1;
104         loc(rt, k);
105         return val[rt];
106     }
107
108     T find_prev(T v) { //查找值v的前驱(小于 v 的最大值)
109         find(rt, v);
110         if (rt && val[rt] < v) return val[rt];
111         int x = ch[rt][0];
112         if (!x) return -1; //返回-1代表不存在
113         for (; ch[x][1]; x = ch[x][1]);
114         splay(rt, x);

```

```

115         return val[rt];
116     }
117
118     T find_next(T v) { //查找值v的后继(大于 v 的最小值)
119         find(rt, v);
120         if (rt && val[rt] > v) return val[rt];
121         int x = ch[rt][1];
122         if (!x) return -1; //返回-1代表不存在
123         for (; ch[x][0]; x = ch[x][0]);
124         splay(rt, x);
125         return val[rt];
126     }
127 };
128 }
129 using Splay_Tree::splay_tree;
130
131 int main() {
132     int q; std::cin >> q;
133     splay_tree t;
134     while(q--){
135         int op, x;
136         std::cin >> op >> x;
137         switch (op) {
138             case 1:
139                 t.insert(x);
140                 break;
141             case 2:
142                 t.remove(x);
143                 break;
144             case 3:
145                 std::cout << t.find_rank(x) << '\n';
146                 break;
147             case 4:
148                 std::cout << t.find_kth(x) << '\n';
149                 break;
150             case 5:
151                 std::cout << t.find_prev(x) << '\n';
152                 break;
153             case 6:
154                 std::cout << t.find_next(x) << '\n';
155                 break;
156         }
157     }
158 }

```

序列操作

Splay 树也可以运用在序列上，用于维护区间信息。与线段树对比，Splay 树常数较大，但是支持更复杂的序列操作，如区间翻转等。

```

1 //文艺平衡树 https://www.luogu.com.cn/problem/P3391
2 #include <iostream>
3
4 constexpr int N = 100005;
5 int id, rt;
6 int fa[N], val[N], sz[N], lz[N], ch[N][2];
7
8 bool dir(int x) { return x == ch[fa[x]][1]; }
9
10 void push_up(int x) { sz[x] = 1 + sz[ch[x][0]] + sz[ch[x][1]]; }
11
12 void lazy_reverse(int x) {
13     std::swap(ch[x][0], ch[x][1]);
14     lz[x] ^= 1;
15 }
16
17 void push_down(int x) { //标记下传
18     if (lz[x]) {
19         if (ch[x][0]) lazy_reverse(ch[x][0]);
20         if (ch[x][1]) lazy_reverse(ch[x][1]);
21         lz[x] = 0;
22     }
23 }
24
25 void rotate(int x) {
26     int y = fa[x], z = fa[y];
27     bool r = dir(x);
28     ch[y][r] = ch[x][!r];
29     ch[x][!r] = y;
30     if (z) ch[z][dir(y)] = x;
31     if (ch[y][r]) fa[ch[y][r]] = y;
32     fa[y] = x;
33     fa[x] = z;
34     push_up(y);
35     push_up(x);
36 }
37
38 void splay(int& z, int x) {
39     int w = fa[z];
40     for (int y; (y = fa[x]) != w; rotate(x)) {
41         if (fa[y] != w) rotate(dir(x) == dir(y) ? y : x);
42     }
43     z = x;
44 }
45
46 void loc(int& z, int k) {
47     int x = z;
48     for (push_down(x); sz[ch[x][0]] != k - 1; push_down(x)) {
49         if (sz[ch[x][0]] >= k) {
50             x = ch[x][0];
51         } else {
52             k -= sz[ch[x][0]] + 1;

```

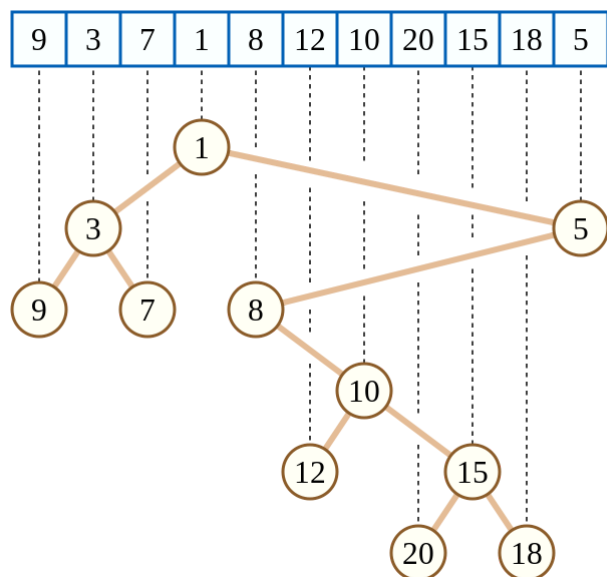
```

53         x = ch[x][1];
54     }
55 }
56 splay(z, x);
57 }
58
59 void build(int n) {
60     for (int i = 1; i <= n + 2; ++i) {
61         ++id;
62         ch[id][0] = rt;
63         if (rt) fa[rt] = id;
64         rt = id;
65         val[id] = i - 1;
66     }
67     splay(rt, 1);
68 }
69
70 void reverse(int l, int r) { // 翻转区间[l,r]
71     loc(rt, l);
72     loc(ch[rt][1], r - l + 2);
73     int x = ch[ch[rt][1]][0];
74     lazy_reverse(x);
75     push_down(x);
76     splay(rt, x);
77 }
78
79 void print(int x) {
80     if (!x) return;
81     push_down(x);
82     print(ch[x][0]);
83     std::cout << val[x] << ' ';
84     print(ch[x][1]);
85 }
86
87 void print_range(int l, int r) { // 输出区间[l,r] (仅验证[1,n]无问题,其它范围未经严谨验证)
88     loc(rt, l);
89     loc(ch[rt][1], r - l + 2);
90     print(ch[ch[rt][1]][0]);
91 }
92
93 int main() {
94     int n, m; std::cin >> n >> m;
95     build(n); // 默认build排列1~n
96     while(m--){
97         int l, r;
98         std::cin >> l >> r;
99         reverse(l, r);
100     }
101     print_range(1,n);
102 }

```

笛卡尔树

笛卡尔树(cartesian-tree)是一种二叉树，每一个节点由一个键值二元组 (k, w) 构成。要求 k 满足二叉搜索树 (BST) 的性质，而 w 满足堆的性质。如果笛卡尔树的 k, w 键值确定，且 k 互不相同， w 也互不相同，那么这棵笛卡尔树的结构是唯一的。如下图：



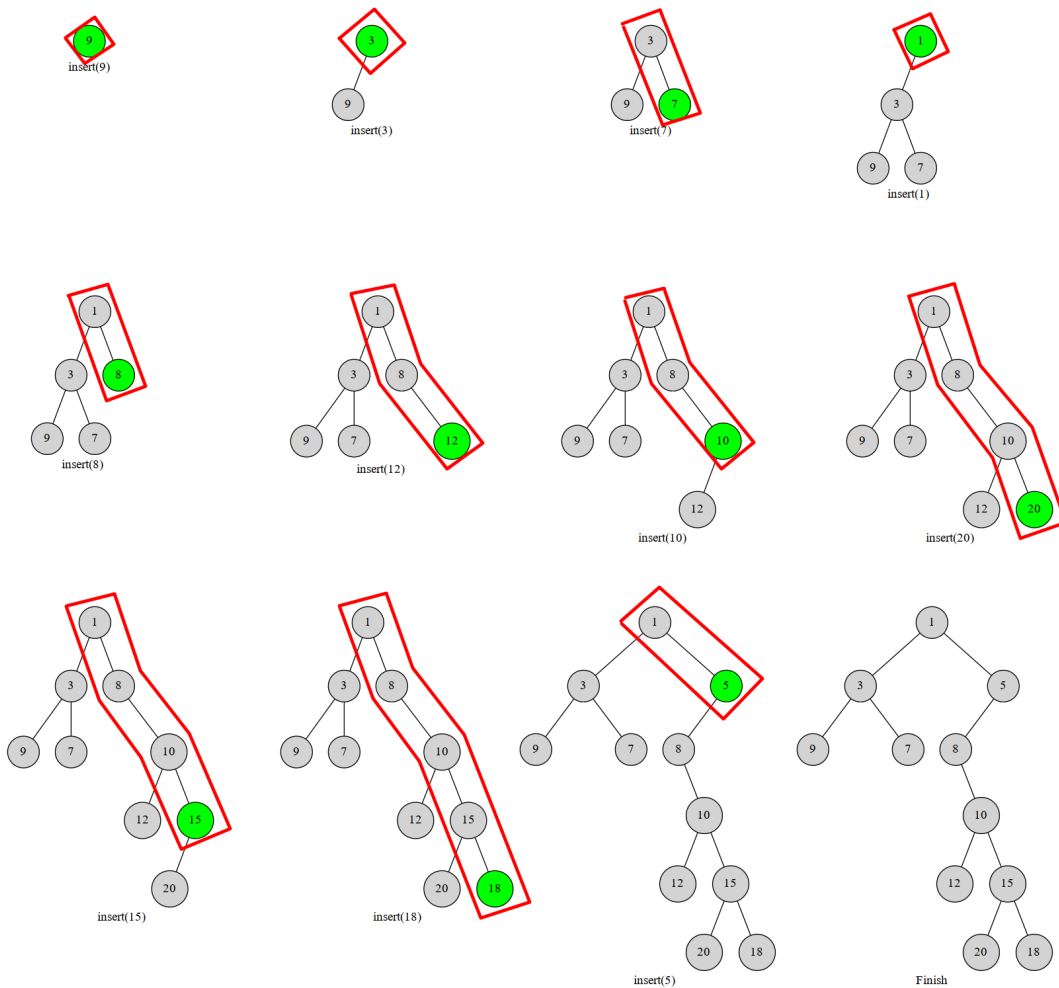
构建过程

我们考虑将元素按 k 升序依次插入到当前的笛卡尔树中。

对于一棵笛卡尔树，定义「右链」为从根节点开始一直走右儿子，走到叶节点形成的链。则插入节点后，这个节点一定在右链上。因为是按照满足 BST 性质的 k 升序插入，那么这个新插入的节点必然在树的 **最右端**。这个节点不可能是一个左儿子，也没有右儿子。

于是我们执行这样一个过程，从下往上比较右链节点与当前节点 u 的 w ，如果找到了一个右链上的节点 x 满足 $w_x < w_u$ ，就把 u 接到 x 的右儿子上，而 x 原本的右子树就变成 u 的左子树。

图中红框部分就是我们始终维护的右链：



```

1 //构建小根堆笛卡尔树 1_idx
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int main () {
6     int n; cin >> n;
7     vector<int> w(n + 1);
8     for (int i = 1; i <= n; i++) cin >> w[i];
9     vector<int> ls(n + 1), rs(n + 1); // 节点i的左右子树节点 ls[i], rs[i], 值为0表示
    不存在
10
11     auto build = [&]() -> int {
12         std::vector<int> stk;
13         for (int i = 1; i <= n; i++) {
14             while (stk.size() && w[stk.back()] > w[i]) ls[i] = stk.back(),
            stk.pop_back();
15             // 当前节点.左儿子 := 最后一个被弹出的节点
16             if (stk.size()) rs[stk.back()] = i; // 栈顶节点.右儿子 := 当前节点
17             stk.emplace_back(i); // 当前节点入栈
18         }

```

```
19         return stk[0]; //返回根节点
20     };
21
22     int root = build();
23     //根据题目要求dfs(root)维护答案
24 }
25
```

一些应用

- 直方图最大矩形/最大全1子矩阵
 - 求所有子区间的最小值之和 / 最大值之和 / 最小值乘长度之和
 - 快速求“左右第一个更小/更大元素”
 - 代替“按最值分割”的分治递归
-

